Arabian Journal for Science and Engineering (2026) 51:6497–6524
https://doi.org/10.1007/s13369-025-10744-3

RESEARCH ARTICLE-COMPUTER ENGINEERING AND COMPUTER SCIENCE

Reinforcement Learning for the Vehicle Routing Problem: Methodologies, Applications, and Research Outlook

Min Liu[1] · Emrah Demir[2] · Ling Jian[1]

Received: 13 February 2025 / Accepted: 9 October 2025 / Published online: 21 October 2025
© King Fahd University of Petroleum & Minerals 2025

Abstract
Recent advances in reinforcement learning (RL) and deep reinforcement learning (DRL) have shown strong potential in generating near-optimal solutions to the NP-hard vehicle routing problem (VRP). This review systematically analyzes 129 relevant studies published between 2015 and 2025 through bibliometric analysis, based on data retrieved from major academic databases such as IEEE Xplore and Web of Science. The search was conducted using keywords such as 'reinforcement learning,' 'traveling salesman problem,' 'vehicle routing problem,' and their variants. The reviewed studies primarily adopt two main approaches: one reformulates the VRP as a Markov decision process and uses DRL to learn its inherent structure; the other integrates RL or DRL with approximation methods such as heuristics, metaheuristics, or hyperheuristics to enhance search efficiency and solution quality. Finally, the challenges of applying RL and DRL methods to the VRP and potential directions for the research are discussed.

Keywords Vehicle routing problem · Reinforcement learning · Heuristics · Metaheuristics · Hyperheuristics

1 Introduction

Reinforcement learning (RL) is a powerful tool for solving the vehicle routing problem (VRP). As a sequential decision-maker, RL can interact directly with the VRP models to provide a generic and flexible framework, which is important in highly dynamic environments that need real-time decisions, such as online takeout and car-hailing services [1–4]. RL can also replace manually designed strategies in operations research (OR) methods by autonomously tailoring their own heuristics to the problem. Especially in the era of a booming e-commerce industry, the integration of RL with powerful deep learning opens up greater possibilities for rapidly and efficiently solving larger and more complex VRP applications. As a result, numerous deep reinforcement learning (DRL) approaches have been developed to address the VRP.

Recent surveys have explored the application of machine learning (ML) techniques for the VRP [1, 5, 6]. These include the use of RL in combinatorial optimization problems, such as maximum cut problem, bin packing problem, and minimum vertex cover problem [7, 8], as well as the integration of RL in intelligent transportation systems, focusing on areas such as traffic signal control and autonomous driving [9, 10]. In recent years, there has been limited attention on the methodological aspects of RL for the VRPs. The work of Raza et al. [11] only summarizes limited number of publications on end-to-end methods. Our paper offers a systematic and comprehensive review, focusing on methodologies and providing clear interpretations of the studies, while identifying common themes, highlighting methodological challenges, and suggesting future research directions.

To conduct this literature review, well-known scientific databases, including Google Scholar, Web of Science, Science Direct, IEEE Xplore, Springer, and ACM Digital Library, have been systematically retrieved, using keywords such as 'vehicle routing problem' or 'traveling salesman problem' and 'reinforcement learning.' A total of 129 papers

✉ Ling Jian
bebetter@upc.edu.cn

Min Liu
upclm@s.upc.edu.cn

Emrah Demir
DemirE@cardiff.ac.uk

[1] School of Economics and Management, China University of Petroleum, Qingdao 266580, Shandong, China

[2] Business and Economics Artificial Intelligence Research Network, Cardiff Business School, Cardiff University, Cardiff, UK

are found, all of which are relevant to the scope of this research. Figure1(left) illustrates the number of publications on VRPs using RL and DRL methods per year. The yearly analysis reveals a noticeable increase in the utilization of RL and DRL approaches for VRP tasks, particularly from 2020 to 2025. These 129 articles can be grouped into two main categories: end-to-end RL and DRL methods, and RL- and DRL-based OR methods, as shown in Fig.1(right).

The first category, termed *end-to-end RL and DRL method*, treats the solution to the VRP as a series of route sequences, each starting and ending at the same depot while sequentially visiting customer nodes. Therefore, the VRP can be viewed as a sequential decision-making problem, making it particularly suitable for solution using RL techniques. Meanwhile, VRP and its variants share a similar internal structure, but only differ in variables and data. It is natural to use DRL approaches to design fully end-to-end neural networks that automatically learn the in-depth combinatorial characteristics [12]. After the learning stage, the model can be generalized and rapidly fine-tuned to further enhance performance and personalization, which is available for stochastic variant instances of similar size in real time without retraining. Thus, the traditional online mathematical programming-based solution development can be transformed into a new offline model training phase [13], which can lighten the computational burden.

The second category, termed *RL- and DRL-based OR methods*, includes enhanced classical OR algorithms like heuristics, metaheuristics, and hyperheuristics for solving the VRP. These OR methods often depend on domain expertise or require a hyperparameter tuning process. During the search process, OR methods generate substantial amounts of data, covering both effective and ineffective solutions, the evolution trajectories of various solutions, and local optima [14]. RL and DRL techniques can improve OR methods by extracting valuable information from the generated data to help reduce the reliance on domain experts or adaptively determine hyperparameters, thereby enabling better decision-making and addressing their inherent limitations.

The remainder of this paper is organized as follows. In Sects. 2 and 3, we present necessary background information, including the VRP formulation, solution algorithms, and RL algorithms. In Sects. 4 and 5, we investigate recent end-to-end RL and DRL approaches and RL- and DRL-based OR methods. Section 6 presents comparative experiments to evaluate different algorithms. Finally, we conclude the review and provide future directions in Sect. 7.

2 The Description of the VRP

To facilitate a better understanding of how RL and DRL are applied to VRP, this section provides a general introduction to the VRP and its main variants.

2.1 The Description of the VRP

The VRP is one of the most well-studied combinatorial optimization problems. It mainly concerns how to efficiently serve a set of customers dispersed geographically around a central depot, using a fleet of homogeneous vehicles [15]. The traveling salesman problem (TSP) is a simplified version of the VRP. It involves no depot, no capacity constraints, and only one vehicle. The multiple TSP (mTSP) extends it by allowing multiple vehicles. The basic VRP, also known as the capacitated VRP (CVRP), requires each node's demand to be met by a single vehicle, with the total demand served by any vehicle not exceeding its capacity. The base model includes a transportation network $\mathcal{G} := (\mathcal{V}, \mathcal{E})$ and a fleet of K vehicles, where $\mathcal{V} = \{v_0, \ldots, v_M\}$ is the set of nodes (locations of depot and customers), and $\mathcal{E} = \left\{l_{ij} : i, j \in \{v_0, \ldots, v_M\}, i \neq j\right\}$ is the set of edges, where l_{ij} denotes the distance between nodes v_i and v_j. And it aims to determine an optimal set of routes (one for each vehicle) that start and end at the depot, collectively covering all nodes and minimizing the total cost (distance). The model can be formulated as follows:

$$\min \sum_{i \in \mathcal{V}} \sum_{j \in \mathcal{V}} x_{ij} l_{ij} \tag{1}$$

subject to

$$\begin{aligned} &\sum_{i \in \mathcal{V}} x_{ij} = 1, \forall j \in \mathcal{V} \backslash \{0\} \\ &\sum_{j \in \mathcal{V}} x_{ij} = 1, \forall i \in \mathcal{V} \backslash \{0\} \end{aligned} \tag{2}$$

$$\begin{aligned} &\sum_{i \in \mathcal{V}} x_{i0} = K \\ &\sum_{j \in \mathcal{V}} x_{0j} = K \end{aligned} \tag{3}$$

$$\begin{aligned} &u_i - u_j + Q x_{ij} \leq Q - d_j, \forall i, j \in \mathcal{V} \backslash \{0\}, i \neq j \\ &d_i \leq u_i \leq Q, \forall i \in \mathcal{V} \backslash \{0\} \end{aligned} \tag{4}$$

$$\sum_{i \notin \mathcal{C}} \sum_{j \in \mathcal{C}} x_{ij} \geq k(\mathcal{C}), \forall \mathcal{C} \subset \mathcal{V} \backslash \{0\}, \mathcal{C} \neq \emptyset. \tag{5}$$

where x_{ij} is a binary variable indicating whether edge (i, j) is included in the solution. $k(\mathcal{C})$ is the least number of vehicles required to serve customer subset. u_i is the load of the vehicle upon arrival at customer node i, Q is the vehicle capacity, and d_j is the demand at node j. Constraints(2) ensure that each customer is visited once and only once, while constraints(3) maintain the correct number of vehicle routes. Constraint(4) eliminate any sub-tours in the routes. Constraint(5) ensures that the demands of all customers are completely fulfilled.

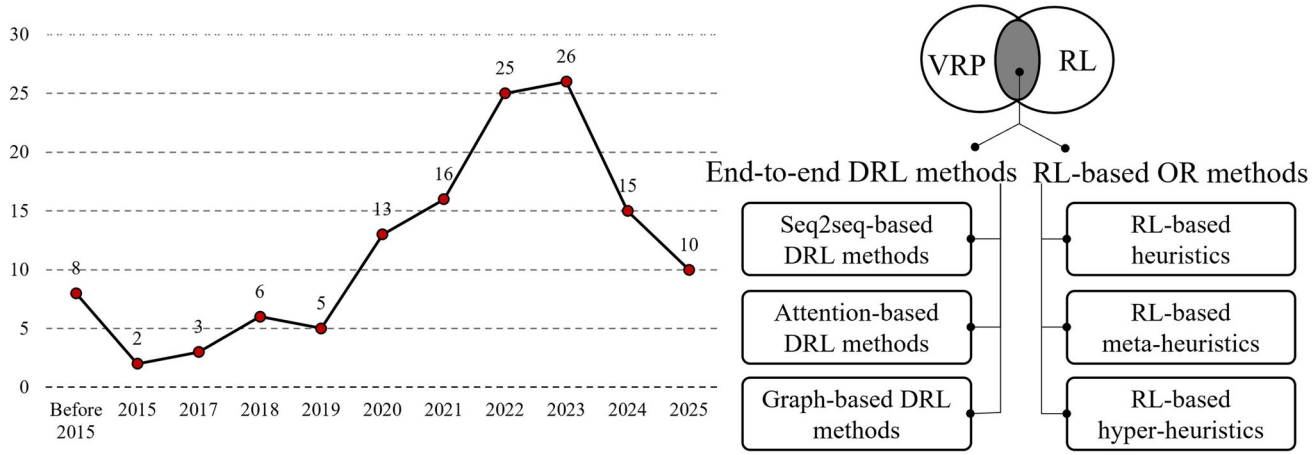


Fig. 1 Number of papers published per year and taxonomy on the use of RL and DRL for the VRPs

2.2 Main VRP Variants

The VRP has many variants. This subsection focuses on those that are addressed in RL and DRL literature on the VRP.

- **HVRP:** In the heterogeneous VRP (HVRP), vehicles in the fleet have different characteristics, such as capacities, speeds, and operating costs.
- **EVRP:** In the electric VRP (EVRP), the fleet consists of electric vehicles (EV), and the node set includes charging stations. Each EV has a limited battery range and must recharge before the battery is depleted.
- **VRPTW:** The VRP with time windows (VRPTW) requires each customer to be served within a specified time interval. Soft time windows allow visits outside these intervals at a cost, reflected as penalties in the objective function.
- **VRPPD:** The VRP with pickup and delivery (VRPPD) requires goods to be collected from specific non-depot origin nodes before being delivered to their designated destinations.
- **DVRP and SVRP:** In real-world applications, the VRP and its variants may be dynamic (DVRP) or stochastic (SVRP), where key parameters (e.g., customer requests or travel times) change over time or are revealed progressively during operations.
- **2EVRP:** The two-echelon VRP (2EVRP) involves two levels of delivery, where goods are first transported from a central depot to intermediate facilities, and then delivered to final customers, reflecting multi-tier distribution networks.
- **MOVRP:** The multi-objective VRP (MOVRP) addresses multiple conflicting objectives simultaneously, such as minimizing total distance, balancing workload among vehicles, and reducing emissions, requiring trade-offs and Pareto efficient solutions.

3 Reinforcement Learning Approaches

RL is a subset of ML focused on learning how to make decisions. It can be formalized by the Markov decision process (MDP), in which an RL agent learns through experience by taking actions a in the environment, inducing changes in the environment's state s, and obtaining rewards r corresponding to actions that evaluate the success or failure of the actions [19]. The goal of the RL is to discover a policy $p(a|s)$ that maximizes the expected sum of discounted rewards. The subsequent subsections present key concepts in RL.

3.1 Policy-Based Methods

Policy-based methods attempt to approximate the policy $\pi(a|s)$ directly and update the parameters θ by gradient ascent in the direction of $\nabla \log \pi_\theta(a|s) L(\theta)$. $L(\theta)$ denotes the estimate of the cumulative reward over a trajectory (where the interaction ends at the time step T) while following the policy $\pi_\theta(a|s)$. And $L(\theta)$ at current time t can be estimated with Monte Carlo method as:

$$L(\theta) = \sum_{i=t}^{T} \gamma^i r(s_i, a_i). \tag{6}$$

In the *Policy gradient*, the training objective of VRP is the expected travel distance, which is defined as $L(\theta) = \mathbb{E}_{p_\theta(\pi|s)}[L(\pi)]$ $(L(\pi) = \mathbb{E}[\sum_{i=1}^{T} r(s_i, a_i)]$, and $r(s_i, a_i)$ is the negative tour length. By the policy gradient theorem [20], the stochastic gradient descent method is applied to optimize the parameter θ in the following form:

$$\nabla L(\theta) = \mathbb{E}_{\pi_\theta(a|s)}[(L(\theta) - b(s)) \nabla \log \pi_\theta(a|s)], \theta \leftarrow \theta - \nabla L(\theta), \tag{7}$$

where $b(s)$ is the baseline function that does not depend on policy $\pi_\theta(a|s)$ and aims to reduce gradient variance, yet keep its unbiasedness. $\nabla L(\theta)$ can update the parameter θ via optimizer.

REINFORCE [21] is a popular policy gradient method. In practice, the expected value in REINFORCE is estimated with Monte Carlo samples in a batch size B:

$$\begin{aligned} \nabla L(\theta) &\approx \frac{1}{B}\sum_{i=1}^{B}[(L(\theta_i) - b(s_i))\nabla \log \pi_\theta(a|s_i)], \\ \theta &\leftarrow \theta - \nabla L(\theta|s). \end{aligned} \tag{8}$$

Actor-critic [22] extends REINFORCE by using a learned state-value function as the baseline, called critic. The critic is parameterized by θ_v, to learn the expected travel distance obtained by current policy $\pi_\theta(a|s)$. And it is trained with gradient descent on a mean squared error objective between its predictions $b_{\theta v}(s_i)$ and the actual tour lengths sampled by the most recent policy [16], defined as:

$$L(\theta) = \frac{1}{B}\sum_{i=1}^{B} \|b_{\theta v}(s_i) - L(\theta_i)\|_2^2 \tag{9}$$

Based on actor-critic, proximal policy optimization *(PPO)* [23] adopts the minibatch update manner to improve data efficiency and performs a clipped surrogate objective $L_{\text{CLIP}}(\theta)$, which serves as a lower bound, to penalize changes to the policy that result in a too large importance sampling ratio $r(\theta)$. The clipped surrogate objective $L_{\text{CLIP}}(\theta)$ is represented as:

$$\begin{aligned} L_{\text{CLIP}}(\theta) &= \mathbb{E}\left[\min\{r(\theta)A, \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)A\}\right], \\ &\text{where} \quad A = L(\theta_i) - b(s_i), \quad r(\theta) = \frac{\pi_\theta(a|s_i)}{\pi_{\theta'}(a|s_i)}, \end{aligned} \tag{10}$$

where A calculates the advantage estimation; $r(\theta)$ represents the probability ratio between two policies; and ϵ is the clip coefficient. As VRP variants become more complex with additional constraints and dynamic elements, the demand for interaction samples in training increases. PPO offers high sample efficiency, allowing effective learning with fewer samples. This makes it well suited for solving large-scale or constrained VRP variants.

3.2 Valued-Based Methods

In contrast to policy-based methods that optimize the optimal policy $\pi_\theta(a|s)$ directly, value-based methods implicitly evaluate the policy by approximating the value function. In VRP, $V_\pi(s_t)$ is the distance (cost) of visiting all remaining customers. Action-value function $q_\pi(s_t, a_t)$ is the expected total distance starting from state s_t, selecting node a_t from the set of yet-to-be-visited nodes, and thereafter reverting to policy $\pi_\theta(a|s)$. The widely used value-based methods include Q-learning and deep Q-network (DQN).

Q-learning is a canonical tabular off-policy algorithm [24]. Its updating rule is defined as:

$$\begin{aligned} q_\pi(s_t, a_t) &\leftarrow q_\pi(s_t, a_t) + \alpha\delta_t \\ \delta_t &= [r + \gamma \max_a q_\pi(s_{t+1}, a) - q_\pi(s_t, a_t)] \end{aligned} \tag{11}$$

where α is the learning rate; γ is the discount factor; and δ_t is the temporal difference (TD) error.

DQN [25] refers to a neural network function approximator with weights θ as the parameter of Q-network. The weights θ can be optimized by minimizing a sequence of loss function $L(\theta_i)$ at each iteration i:

$$\begin{aligned} L(\theta_i) &= \mathbb{E}_{s_t, a_t, r, s_{t+1}}[(y_i - q(s_t, a_t; \theta_i))^2] \\ y_i &= r + \gamma \max_a q(s_{t+1}, a, \theta^-). \end{aligned} \tag{12}$$

The main advantages of DQN are: (i) using the experience replay to eliminate the sample correlations and (ii) freezing the target network $q(s_{t+1}, a_{t+1}, \theta^-)$ to stabilize the learning of Q-function while updating the online network $q(s_t, a_t; \theta_i)$ by stochastic gradient descent via

$$\nabla_{\theta_i} L(\theta_i) = \mathbb{E}_{s_t, a_t, r, s_{t+1}}[(y_i - q(s_t, a_t, \theta_i))\nabla_{\theta_i} q(s_t, a_t; \theta_i)]. \tag{13}$$

Further advancements in DQN have led to the development of several more sophisticated methods, such as double DQN [26], which integrates double Q-learning [27] with DQN to mitigate overestimation issues, and dueling DQN [28], which separates the state value from the action advantage function to enhance performance. Double DQN and dueling DQN reduce bias, improve stability, and enhance policy quality compared to basic DQN, making them potentially better suited to learn the VRP's structure and obtain improved solutions.

4 End-to-end RL and DRL Methods

This section presents existing end-to-end RL and DRL approaches that integrate deep structures such as sequence-to-sequence (seq2seq) paradigms, attention mechanisms, and graph neural networks (GNNs).

4.1 VRP as Markov Decision Process

The VRP can be formulated as a MDP for the application of RL in model optimization. The tuple {S,A,T,R,P} of

Table 1 Markov decision process for the VRP and its variants

Problem	Static state	Dynamic state	Action	Reward	Reference
TSP/mTSP	Coordinates	-	Next move	Additional distance	[16, 33] [31, 34] [35, 36]
VRP	Coordinates, demands	Last visited node, vehicle' remaining capacity, partial route of the vehicle	Next move	Additional distance	[33, 37] [31, 38] [35, 36]
MOVRP	Coordinates,demands, objectives' weight	Last visited node, vehicle' remaining capacity, partial route of the vehicle	Next move	Additional distance	[39]
EVRP	Coordinates (charging station), demand	EV' load,battery,traveled time, the number of vehicles available (not in charging)	Next move (customer, depot or charging station)	Additional energy consumption	[40, 41]
VRPTW	Coordinate,demands,time window,penalty coefficient	Vehicle' remaining capacity, installer' service capacity	Next move	Additional distance/time (+penalty term)	[17, 29] [42]
DVRP	Coordinates,demands, traffic patterns	Current customer pool, visited status of customers, current traffic conditions	Next move	Additional time	[43]
HVRP	Coordinates,demands, vehicle capacity	Last visited node, vehicle' remaining capacity, partial route of the vehicle	Select a vehicle and a node	Incremental time for all vehicles	[44]
VRPPD	Coordinates(pickup,delivery) demands(pickup,delivery)	Partial route of the vehicle	Next pickup or delivery	Incremental time	[45]

the MDP (state, action, transition, reward, and policy) of the basic VRP is defined as follows (Table 1).

State: Most of the end-to-end RL and DRL algorithms for the VRP model the agent as a vehicle with environment perception [9]. The global state can be divided into two components: (i) the dynamic state, encompassing the current vehicle position and its remaining capacity and a partial solution (the set of all previously visited customers); and (ii) the static state, comprising customers' demand and the coordinates of the depot and customers. The dynamic state may change at each iteration, while the static state remains constant throughout the entire process.

Action: The action of the agent is to specify the next move (customer or depot) for the current vehicle. The permutation of consecutive actions forms a complete route $\pi = (a_1 \rightarrow a_2 \rightarrow, ...a_t \rightarrow ...)(a_t \in \{v_0, \ldots, v_M\}; a_t \neq a_{t'}, \forall t \neq t')$. Already visited customers are irrelevant to future decisions. Therefore, their log probabilities are set to zero by the mask to avoid overlapping selection. In multi-vehicle problems where decisions for all vehicles are made at each time step, multiple agents make decisions sequentially to prevent conflicting actions, and different routes are distinguished by the insertion of depots [29].

Transition: The transition involves adding the last visited customer to the partial solution and updating the state of both the corresponding customer (its demand becomes zero once visited) and the vehicle (its location and residual capacity both change).

Reward: The reward is usually related to the objective function Eq.(1). In the case of the VRP aiming to minimize total travel distance, the reward is defined as the difference in the distance functions after transitioning from state s_t to s_{t+1} when taking a specific action a_t, i.e., $r(s_t, a_t) = -\|L(\pi_t) - L(\pi_{t-1})\|_2$, where $\pi_t = (a_1 \rightarrow a_2 \rightarrow, \ldots, \rightarrow a_t)$ is the partial solution at time t, $L(\pi_t)$ is it's length. Consequently, the cumulative reward of a terminal state corresponds precisely to the total length of the route. For VRP with soft time windows, a negative penalty is added to the reward when time constraints are violated.

Policy: Policy $p_\theta(\pi|s)$ refers to the mapping from perceived states to a probability distribution over possible actions at any time step t. It can be factorized as the probability chain rule: $p_\theta(\pi|s) = \prod_{t=1}^{M} p(a_t|s_t, a_{1:t-1})$. DRL methods aim to minimize the reinforced loss by optimizing the policy $p_\theta(\pi|s)$ based on its experience. The loss function is defined as the difference between the expected travel distance $L(\pi)$ and the baseline $b(s)$. $b(s)$ is performed by a critic network to reduce gradient variance while preserving unbiasedness. To obtain the optimal solution for the VRP, stochastic gradient descent [30] is used to optimize θ in the following form:

$$\nabla L(\theta|s) = \mathbb{E}_{p_\theta(\pi|s)}\left[\left(L(\pi) - b(s)\right)\nabla log p_\theta(\pi|s)\right], \tag{14}$$

where $L(\pi) = \sum_{t=1}^{M} \gamma^t r(s_t, a_t)$ is the cumulative reward function, and the discounting factor γ^t is typically set to one in VRP. Adam is used to update the parameter θ.

4.2 Seq2seq-based RL and DRL Approaches

Seq2seq-based learning methods employ an encoder–decoder structure, the encoder extracting features from the input instances, and the decoder iteratively constructing the solution. Throughout the construction of each solution sequence, the decoder predicts the probability distribution for selecting each node and adds a node to the end of the current generated sequence until completion [31].

Network structure: The seq2seq paradigm [32] is a powerful model proposed in natural language processing. It uses one recurrent neural network (RNN) encoder to map an input sequence into a high-dimensional vector and another RNN decoder to unfold the obtained embedding into an output sequence. This model introduces new potential for addressing sequential problems, such as the VRP. However, the seq2seq framework cannot be applied directly in the VRP, because it is associated with a fixed output dictionary, while the VRP has a varying output permutation [46]. Vinyals et al. first overcame the above restriction by introducing the pointer network (PtrNet), which connects the seq2seq with a pointer mechanism to autoregressively select a specific position with highest conditional probability from the input elements as output [47]. This structure is invariant to the input length and guarantees that the output sequence is determined by the input. The solving process of the vanilla PtrNet model is as follows.

For a TSP instance with M customer nodes, an RL episode consists of two phases: the encoding process at the beginning and the decoding process over M time steps to yield the node permutation $\pi = (a_1 \rightarrow a_2 \rightarrow, \ldots, \rightarrow a_M)$ as the output. First, the RNN-based encoder is sequentially fed with some important factors $\{s_i\}_1^M$ from nodes $\{v_i\}_1^M$, and transforms them into a sequence of latent memory states $\{h_i\}_1^M$ until the end of the input sequence is reached, at which point a special symbol $\rightarrow$ is input to the model, as illustrated in Fig. 2. At each time step t, the RNN in the decoder reads the last latent memory d_{t-1} and the feature of the last visited node $h_{a_{t-1}}$, and computes its current latent memory d_t (at time step 1, the input to the decoder is the output of the encoder). Based on d_t and the features $\{h_i\}_1^M$, the probabilities of selecting each node are determined using the pointer attention mechanism as follows.

$$\begin{aligned} u_j^t &= \kappa^\intercal(W_1 h_j + W_2 d_t), \; j \in \{1, \ldots, M\} \\ p(a_{t+1}|a_1, \ldots, a_t, \pi) &= \text{softmax}(u^t), \end{aligned} \tag{15}$$

where softmax function normalizes u^t into an output distribution, and κ, W_1, W_2 are learnable parameters of the decoder.

Exhaustive, sampling, greedy, or beam search strategies ultimately determine the node to which the pointer points. Upon traversing each node exactly once, the agent finalizes the permutation π by returning to the initial node. Policy gradient methods and stochastic gradient descent are employed to optimize the parameters of the seq2seq model. The gradient is formulated using RL approaches, as discussed in Sect. 3.

Application: Among recent learning-based models for construction heuristics, PtrNet is the initial foundation. Vinyals et al. introduce PtrNet to tackle TSP and train it in a supervised manner with pairs of TSP instances and solutions from approximate solvers as labels [47]. This supervised learning (SL) technique may be advisable when ground-truth data or labels are available but may be a drawback in cases where data collection is not feasible. Hence, Bello et al. [16] apply the actor-critic approach, treating each TSP instance as a training sample, and utilizing the tour length of a sampled solution for an unbiased Monte Carlo estimate of the policy gradient. Their results demonstrate the potential of RL for addressing this type of problem. However, this approach cannot be applied directly to the VRP because it assumes that the system remains static over time. Actually, in the VRP, the vehicle's remaining capacity changes once a node has been visited. To address this issue, Nazari et al. [37] propose a RNN decoder coupled with an attention mechanism. At each time step, the embeddings of the static elements are fed into the RNN decoder, and the output of the RNN and the dynamic element embeddings are input into the attention mechanism. This design ensures that the static sequence is processed only once in the encoder, while the dynamic sequence is updated at each time step. Since the final output is independent of the input order, they simplify the RNN-based encoder with an embedding layer, omitting positional information, and reducing computational complexity. Zhao et al. [17] expand the work of [37] by creating a routing simulator to generate extensive VRP instances for training, along with implementing local search methods to enhance solution quality. The summary of the above methods can be found in Table 2.

4.3 Attention-based RL and DRL Approaches

Network structure: Seq2seq models are helpful in tasks that involve mapping from one sequence to another. But since the coordinates of the customers do not convey sequential information, the procedure of solving VRP is actually a set-to-sequence prediction problem where the input is a set of customers and the output is a permutation of the input [50]. Motivated by the success of transformer [52], Deudon et al. [33] and Kool et al. [30] replace the seq2seq model with a transformer-like encoder–decoder architecture. Meanwhile, the encoder is order-invariant by excluding positional embedding in the vanilla transformer [52]. The structure of this model is represented as follows.

The encoder comprises an embedding layer and N identical attention layers. Each layer consists of a multi-head attention (MHA), a fully connected feed-forward (FF) network, a skip connection (SC), and batch normalization (BN), as shown in Fig. 3(A). The attention mechanism is a weighted message passing approach in which the weight is based on the compatibilities between two arbitrary elements in the input. Multi-head attention is used for feature augmentation, injecting information jointly from different representation subspaces at various positions. The static feature s_i for each vertex in the VRP instances is denoted as x_0 for the depot and $[x_i, d_i]$ for the customer v_i (x_i is a two-dimensional coordinate and d_i is the normalized demand). The encoder computes initial node embeddings $\{h_i^0\}_{i=0}^M$ through a learned linear projection, with $h_0^0 = W_0 x_0 + b_0$ for the depot and $h_i^0 = W[x_i, d_i] + b$ for customers. The embeddings are updated using N attention layers, as follows.

$$\begin{aligned} \hat{h}_i &= BN^l\left(h_i^{l-1} + \mathrm{MHA}_i^l(h_0^{l-1}, \ldots, h_M^{l-1})\right) \\ h_i^l &= BN^l\left(\hat{h}_i + \mathrm{FF}^l(\hat{h}_i)\right). \end{aligned} \tag{16}$$

Finally, the encoder computes an aggregated embedding h^N of all customers as the mean of the final node embeddings, as shown in Fig. 3(C). These node embeddings remain fixed throughout the episode. The node embeddings h_i^N and the embedding h^N are used as input to the decoder.

The decoder constructs the solution iteratively, one node at a time, as depicted on the left side of Fig. 3. It contains a multi-head self-attention and a pointer network, as shown in Fig.3(B). During decoding, the state representation comprises two parts: all node embeddings $\{h_i^N\}_{i=0}^M$ and the current context embedding h_c^N which captures the state changes. For the TSP, the context embedding consists of the embedding h^N, the last visited node a_{t-1} and the first node a_1. For the VRP, the context embedding contains h^N, a_{t-1} and the remaining vehicle capacity D. The MHA computes a new context embedding h_c^{N+1} by passing messages only from node embeddings to the context node. The pointer network calculates the output probabilities. For this layer, the compatibility between h_c^{N+1} and node v_i is computed as.

$$u_i = \begin{cases} \mathrm{C} \cdot \tanh\left(\dfrac{\left(w_q h_c^{N+1}\right)^{\mathsf{T}} \cdot \left(w_k h_i^N\right)}{\sqrt{d_k}}\right), & \text{if } v_i \notin \pi_{t'},\ t' \le t, \\ -\infty, & \text{otherwise,} \end{cases} \tag{17}$$

where tanh is used to clip the result within [-C, C], d_k is used for scalarization, and $\pi_{t'}$ is the partial solution at time t'. The second line of Eq.(17) illustrates a visitation mask, which is used to track visited nodes and avoid their reselection. The

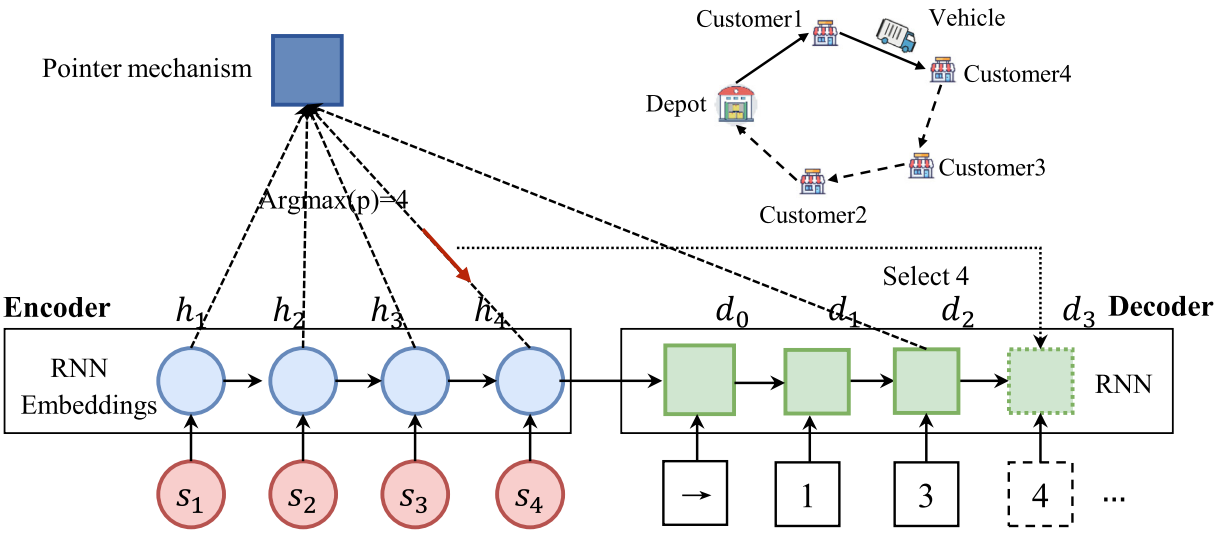


Fig. 2 Schematic diagram of the seq2seq-based model

Table 2 Summary of seq2seq-based RL and DRL approaches

Reference	Problem	Encoder	Decoder	Training method	Instance size
[47]	TSP	LSTM	LSTM+PtrNet	SL	50
[16]	TSP	LSTM	LSTM+PtrNet	REINFORCE	20, 50, 100
[37]	VRP	LA	RNN+Att+PtrNet	Policy gradient	10, 20, 50, 100
[17]	VRP,VRPTW	LA	LSTM+Att+PtrNet	Policy gradient	20, 50, 100
[49]	SVRP	LSTM + CNN	LSTM+Att+PtrNet	RL	10, 20, 50, 100

LSTM = Long short-term memory; Att = attention; LA = linear affine; A2C = advantage actor-critic; CNN = convolutional neural network

ultimate probability of choosing the next node v_i is defined as $p_i = \frac{e^{u_i}}{\sum_j e^{u_j}}$.

Application: Deudon et al. [33] propose a transformer-like architecture to improve PtrNet [47] by achieving invariance over the input order. This model encodes all node coordinates into a corresponding number of action vectors using the transformer structure and decodes the last three actions to derive a query vector. The policy network is trained with actor-critic to autoregressively construct TSP solutions. They also incorporate a 2-opt-based local search to further enhance performance. Kool et al. [30] extend the encoder–decoder framework with multi-head attention layers to tackle the VRP and split delivery VRP, and use a reinforce gradient estimator with a deterministic greedy rollout baseline to train the model. The deterministic greedy rollout is generated based on the current optimal policy model. This strategy is more efficient than the work of Deudon et al. [33]. Since then, a lot of similar transformer-based methods have emerged to boost algorithms' performance or adapt to various variants by adjusting network architecture or training procedure. Kwon et al. [53] enhance attention mechanisms by sampling multiple trajectories in parallel. Xin et al. [31] argue that the solutions created using only one policy lack diversity. Hence, they propose a multi-decoder attention model to enhance the likelihood of discovering good solutions by sampling different policies, where the Kullback–Leibler divergence is included to ensure policy diversity. Xu et al. [38] posit that tracking only the first and last selected nodes in the context embeddings is insufficient to represent the dynamics of state. Therefore, they introduce an attentive aggregation module in the decoder to build a dynamics-aware context embedding with multiple relational structures: unvisited graph embedding and visited graph embedding. Kim et al. [54] train a constructive policy to generate diverse candidate solutions, followed by an improvement policy that iteratively refines these solutions to enhance the solution quality for TSP and VRP. Zong et al. [55] introduce a rewriting-by-generating framework for solving large-scale routing problems. The framework generates regional routing solutions within each independent region using an attention-based generator and rewrites its previous solution by merging and repartitioning region pairs via a REINFORCE-based rewriter. Tang et al. [41] extend the attention-based DRL approach to energy minimization electric VRP by adding a feature embedding module between the encoder and the decoder to better cap-

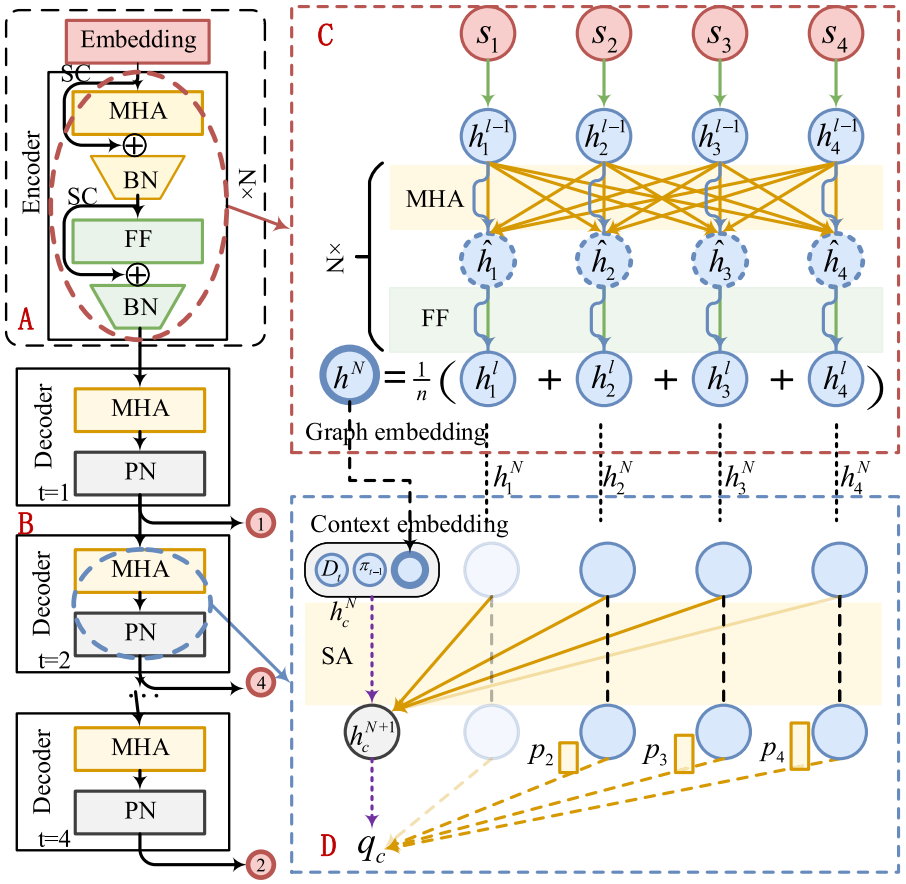


Fig. 3 Schematic diagram of the attention-based encoder–decoder architecture

ture the state and routing features of EVs better. Zhang et al. [43] propose a DRL method with two encoder–decoder architectures (M1 and M2) to solve dynamic TSP with changing customers and traffic conditions. M1 encodes once and decodes at each time step to handle dynamic traffic, similar to the approach of Kool et al. [30]. M2 encodes and decodes at each time step to address dynamic customer requests. Bono et al. [56] solve the dynamic VRP by incorporating two MHA layers into the encoder, enabling it to track the state of all vehicles and adapt to the dynamic environment. Ren et al. [57] adopt a multi-agent framework, treating each vehicle as an agent to solve the multi-vehicle routing problem. Zong et al. [58] enhance an attention-based DRL framework with two key improvements to tackle the multiple vehicle routing problem with time windows. They add a penalty to the reward function to handle time window constraints and implement a two-step strategy: first determining the optimal customer visit sequence, then assigning customers to vehicles based on capacity limits. The summary and comparison of the aforementioned and more similar methods can be found in Table 3.

4.4 Graph-assisting RL and DRL Approaches

Network structure: The VRP can be represented as a complete undirected graph due to its highly structured nature. Taking full advantage of this intrinsic graph property is essential for constructing high-quality solutions. Therefore, GNN can be a powerful tool for VRP, using its capability in information embedding and belief propagation within the graph topology [64]. A GNN framework consists of a stack of neural network layers, with each layer aggregating local neighborhood information based on the graph's topology, as follows:

$$\mu_v^{(t)} \leftarrow f(\mu_v^{(t-1)}, \{\mu_u^{(t-1)}\}_{u\in\mathcal{N}_v}, \{w(v,u)\}_{u\in\mathcal{N}_v}; \theta), \quad (18)$$

where $\mu_v^{(t)}$, $\mu_v^{(t-1)}$ are the features of node v at step t and $t-1$, respectively; $\{\mu_u^{(t-1)}\}_{u\in\mathcal{N}_v}$ are the features of neighbors at step $t-1$; $\mathcal{N}_v$ is the set of neighbors of node v; $w(v,u)$ is the weight of edge (v,u); f is a generic nonlinear mapping, θ is the parameters of f. After a few recursions, GNN generates a new embedding for each node by considering both graph characteristics and long-range interactions between these node features [81]. Unlike RNNs and attention mechanisms, GNN integrates edge information into each update layer, which can enhance feature extraction and improve generalization for large-scale instances.

Application: Khalil et al. [81] employ fitted Q-learning to learn a vertex selection policy parameterized by the graph embedding network, known as structure2vec (S2V). An insertion algorithm helps compute the most suitable posi-

Table 3 Summary of attention-based RL and DRL approaches

Reference	Problem	Encoder	Decoder	Training method	Problem size	Description
[33]	TSP	MHA	LA+PtrNet	Policy Gradient	20,50,100	-
[30]	TSP,VRP	MHA	MHA+PtrNet	REINFORCE	20,50,100	-
[59]	VRP	MHA	MHA+PtrNet	REINFORCE	20,50,100	Encoder and decoder are used alternately to recode the embedding of each node and construct a partial solution.
[29]	VRPTW	MHA	MHA+PtrNet	Policy gradient	20,50,100,150	-
[60]	VRPTW	MHA	MHA+PtrNet	REINFORCE	20,50	Add two additional encoders for tours
[31]	TSP,VRP	MHA	MHA+PtrNet	REINFORCE	20,50,100	Use multi-decoder to enhance the chance of finding good solutions
[61]	TSP,VRP	DACT	MHA+PtrNet	PPO	20,50,100	Learn separate embeddings for node and positional features
[56]	DVRP	MHA	MHA+PtrNet	Actor-critic	20,50,100	-
[45]	VRPDPD	HMHA	MHA+PtrNet	REINFORCE	-	Design a multi-attention mechanism to learn different relationships between ego pickup point, ego delivery point and other points.
[62]	MDVRP	MHA+AOA	MHA+AOA+PtrNet	Actor-critic	-	Add an attention-to-attention mechanism in the encoder and decoder to improve the decoder's accuracy by considering the correlation between query vectors and query results in MHA.
[63]	HVRP	MHA	MHA+PtrNet FF+LA+PMPtrNet	Policy gradient	40-160	Add a extra decoder to select a vehicle before a node selection.
[42]	EVRPTW	CNN+MHA	LSTM+PtrNet	REINFORCE	50	Add a one-dimensional convolutional layer for considering global information.
[38]	TSP,VRP	MHA	MHA+PtrNet	Policy gradient	20,50,100	Build a dynamics-aware context embedding to capture state changing.
[64]	TSP,VRP	EMHA	MHA+PtrNet	REINFORCE	20,50,100	Take into consideration the edge information and residual connections between layers.
[12]	DIRP	MHA	BS +PtrNet	Actor-critic	10,20,50,100	Replace MHA with MHCA in the encoder to reduce computational complexity.
[57]	MVRPTW	GRU+MHA	MHA+PtrNet	Policy gradient	20,50,100	Use GRU to capture historical route information of vehicles
[55]	VRP	MHA	MHA+PtrNet	REINFORCE	500,1000,2000	Use rewriting-by-generating framework for solving large-scale routing problems.
[65]	VRPDP	MHA	MHA+PtrNet	Cooperative A2C	20,50,100	Design a paired context embedding to capture the interdependencies of heterogeneous nodes.

Table 3 continued

Reference	Problem	Encoder	Decoder	Training method	Problem size	Description
[66]	MDVRP	MHA	MHA+PtrNet	Policy gradient	50,100	Design an encoder to extract the relationships among the pickup and delivery customers.
[36]	TSP,VRP	MHCA	MHCA+PtrNet	REINFORCE	20,50,100	Add a feature embedding to capture the EVs' state
[41]	EVRP	MHA	MHA+PtrNet	Policy gradient	-	-
[39]	MOTSP	MHA+Att	MHA+PtrNet	REINFORCE	100,150,200	Set multi-decoders to learn different problem-solving strategies.
[67]	TSP,VRP	MHA+GRU +AGA+GNL	MHA+PtrNet	A2C	100,150,200	Add a refiner to dynamically improve the feature embeddings from the encoder.
[43]	DVRP	MHA	MHA+PtrNet	Policy gradient	100	Add another encoder–decoder framework to address dynamic customer requests.
		MHA	MHA+PtrNet			
[68]	MTSP	Gated transformer	MHA+PtrNet	REINFORCE	50,100	A agent represents a traveler
[51]	VRP	BERT	PtrNet	Contrastive learning	20,50,100	Use transformer encoder of BERT due to its more robust encoding capability.
				Hierarchical REINFORCE		
[69]	TSP	MHA	MHA+PtrNet	REINFORCE	20,50,100	Build dynamics-aware context embedding by considering node visitation information.
[70]	VRP	MHA	MHA+PtrNet	REINFORCE	20,50,100	Adopt contrastive self-supervised learning to supervise the quality of the node embedding extracted by encoder at each step.
[71]	TSP	MHA	MHA+PtrNet	REINFORCE	20,50,100, 200-1000	Replace scaled dot product attention with sampled scaled dot product attention in encoder and eliminate masked MHA in decoder to reduce time and space complexity.
[40]	MDVRP	MHA	MHA+PtrNet	REINFORCE	20,50,100	Multiple agents work together
[58]	MVRPTW	MHA	MHA+PtrNet	REINFORCE	20,50,100	Design a time window penalty augmented reward and a two-step strategy.
[72]	VRP	MHA	MHA+PtrNet	REINFORCE	50,100	Combine ensemble learning to obtain sub-policies for handling various distributions.

Table 3 continued

Reference	Problem	Encoder	Decoder	Training method	Problem size	Description
[73]	HVRP	MHA	MHA+PtrNet	Policy gradient	20,50,100	Use separate encoders to update dynamic state of vehicles and environment at each step.
[74]	VRPDP	MHA	MHA+PtrNet	REINFORCE	20,50,100,150	Design a two-stage attention-based encoder to distinguish delivery customers, pickup customers, and the depot.
[75]	VRPDP	GAT	MHA+PtrNet	REINFORCE	10,20,50,100	Use a graph attention network-based encoder to encode node and edge information.
[76]	EVRP	MHA	MHA+PtrNet	PPO	10,20,50,100	Design a constructor to capture dynamic state of EV at each step.
[77]	TSP, VRP	SAT	Max pooling+PtrNet	PPO	20,50,100,1000	Use synergetic attention mechanism that learns node and index features via two paths.
[78]	EVRP	MHA	MHA+PtrNet	DRL	10,20,50	An embedding enhancement module is incorporated to improve node representations.
[79]	VRP,VRPTW	multi-MHA	multi-MHA+PtrNet	REINFORCE	50,100	Multiple MHAs are employed to learn general patterns and individualized preferences.
[80]	2EVRP	MHA	MHA+PtrNet	Policy gradient	100,200,300	Use DRL to solve two-echelon VRP.

DACT = Dual aspect collaborative transformer; PPO = proximal policy optimization; FF = feed-forward network; HMHA = heterogeneous MHA; AOA = attention-to-attention mechanism; EMHA = edge MHA; BS = beam search; GRU = gated recurrent unit; MHCA = multi-head cosine-based attention; AGA = accumulated graph attention; GNL= gated nonlinear layer; BERT = bidirectional encoder representations from transformers; GAT = graph attention network; SAT = synergetic attention-driven transformer

Table 4 Summary of GNN-based RL and DRL approaches

Reference	Problem	Network	Training method	Instance size
[81]	TSP	Graph embedding	fitted Q-learning	50-100
[82]	TSP	GCN	SL	20,50,100
[50]	CARP	GCN, RNN encoder and decoder, GRU encoder and decoder	REINFORCE	-
[84]	Online VRP	S2V+ LSTM encoder and decoder	REINFORCE	-
[85]	mTSP	CMPNN	S-Samples batch REINFORCE	
[86]	VRP	GCN encoder, MLP decoder, GRU decoder	SL+REINFORCE	100,200,400
[87]	TSP,VRP	GAT,GRU	Q-learning	500,750,1000
[83]	TSP	BGNN	Imitation learning	200
[88]	DVRP	GNN, LSTM encoder and decoder	A3C	20,50,100,200
[89]	mTSPTW	GIN, attention-based encoder and decoder	REINFORCE	50,100-500

CMPNN = Compositional message passing neural network; MLP = multilayer perceptron; BGNN = bidirectional GNN; GIN = graph isomorphism network; A3C = asynchronous A2C

tion for the vertex in the partial solutions. Using pairs of TSP instances and optimal solutions obtained from the Concorde TSP solver as labels, Joshi et al. [82] train graph convolutional network (GCN) in a supervised manner to directly generate a tour as an adjacency matrix, which is subsequently converted into a feasible solution using beam search. Li and Li [50] employ a graph convolutional network (GCN) to analyze the information graph of the capacitated arc routing problem (CARP) and utilize two separate encoder–decoder models: one to generate the initial solution and the other to indicate the directions of the edges in the solution. The two policies are, respectively, parameterized by the RNN network and gated recurrent unit (GRU) network, and are simultaneously trained using the REINFORCE algorithm. Hu et al. [83] employ a bidirectional graph neural network (BGNN) in combination with imitation learning to address sparsely connected TSP. The summary and comparison of the aforementioned and more similar methods can be found in Table 4.

4.5 Multi-objective Optimization Strategy

Building on the above single-objective end-to-end RL and DRL methods, several multi-objective approaches have also emerged. Decomposition is a mainstream strategy for addressing multi-objective optimization problems, such as those discussed in [90], Wu et al. [91] for the MOTSP and Zhang et al. [92] for the MOVRPTW. Although these algorithms can outperform traditional methods, they lack flexibility in handling new weight vectors. Specifically, the weight vectors for decomposing problems are predetermined before training, restricting the trade-off solutions to be obtained solely by solving the subproblems with the fixed weight vector [39]. In addition, the size of the Pareto front grows exponentially with respect to the problem scale, and training numerous sub-models for large-scale problems becomes time-consuming. Hence, Lin et al. [34] propose a single preference-conditioned attention model to directly learn the Pareto front. They employ completely different decoders for distinct objective weight vectors, with the parameters of these decoders determined by a hypernetwork. Ye et al. [35] also focus on approximating the Pareto front. They incorporate a weight-adaptive layer in the decoder to capture guided information for different objective weight vectors. Zhang et al. [93] train another metamodel to quickly adapt to a sub-model for the unseen subproblem, requiring only a few fine-tuning steps. Li et al. [39] treat the weight vectors as input and employ multiple decoders to learn various problem-solving strategies. The summary and comparison of the above can be found in Table 5.

4.6 General Observations and Insights

In addition to the seq2seq-based, attention-based and graph-based approaches, researchers formulate the VRP as a MDP and train it using RL or DRL approaches with CNN [97, 98] or other NNs, such as the work of [19, 99–110]. It is worth noting that most learning-based methods train and test deep models on the same instance distribution. Therefore, some cross-distribution methods are proposed to generalize the learned policy to other distributions. Jiang et al. [97] use CNN to extract distribution-aware features and adopt group Distributionally Robust Optimization to strengthen model robustness. Zhang et al. [111] develop a hardness-adaptive TSP instance generator and used curriculum learning to train the model. Bi et al. [112] propose an adaptive multi-distribution knowledge distillation scheme to

improve cross-distribution generalization in VRPs by unifying diverse policies from multiple distributions into a single, robust model. Jiang et al. [72] use an ensemble-based DRL method for VRPs, training diverse sub-policies to address various instance distributions.

Throughout the literature review, end-to-end approaches share some common patterns despite different model designs. These include: (i) an embedding layer is usually indispensable to map the problem-specific information; (ii) most of the literature follows an encoder–decoder architecture that employs the pointer mechanism to predict the conditional probability of customers and autonomously construct routes in an autoregressive manner until termination. The embeddings of the static elements are fed into the encoder, and the context embeddings, which concatenate the output of the encoder and the dynamic element embeddings, are input into the decoder. This can ensure that the static sequence is processed only once in the encoder, while the dynamic sequence is updated at each time step. (iii) As the customer's coordinates do not provide sequential information, attention-based and graph-based methods have a broader range of applications compared to seq2seq-based methods. Graph-based methods are more suitable for solving large-scale instances. (iv) The end-to-end approaches for single-objective optimization can extend to multi-objective problems by employing decomposition strategies, training another weight-tuning model, or using multiple decoders for distinct objective weight vectors. (v) Training methods can be broadly categorized into supervised learning, RL, or a combination of both to form a two-stage joint training model. RL emerges as the superior performer, freeing itself from the challenges associated with obtaining high-quality labels and offering better generalization compared with supervised learning. (vi) Due to the complexity of most VRP and its variants, obtaining an accurate system description function is challenging, leading to the prevalent use of model-free RL. The REINFORCE and policy gradient algorithms are widely employed in VRP literature because most researchers adopt a policy-based learning approach, constructing routes based on the conditional probability of customers. Hierarchical and multi-agent reinforcement learning methods are more suitable for large-scale problems. (vii) The majority of end-to-end methods fall into the category of construction heuristics, where they generate a sequence of solutions from scratch, one by one. End-to-end RL and DRL approaches present a novel direct solution for VRP, showing a range of advantages over OR methods.

Strong generalization: OR methods find problem-specific solutions by considering each instance separately, limiting their adaptability and flexibility. In contrast, DRL algorithms interact with the VRP models and exploit inherent patterns to develop heuristics. Once trained, DRL algorithms can address VRP as a sequence of consecutive actions in real

Table 5 Summary of multi-objective RL and DRL approaches

Reference	Problem	Model	No. of Models	Training methods	Instance size
[90]	MOTSP	CNN encoder and GRU decoder	No. of preferences	Actor-critic	40,70,100,150,200
[91]	MOTSP	Attention encoder and decoder	No. of preferences	REINFORCE	40,70,100,150,200
[92]	MOVRPTW	MHA,MLP encoder & MHA,PtrNet decoder	No. of preferences	REINFORCE & EL	50,80,100
[34]	MOTSP	Hypernetwork and attention encoder–decoder framework	1	REINFORCE	20,50,100
[35]	MOTSP,MOVRP	MHA encoder & weight-specific decoder	1	REINFORCE	100,200,500
[93]	MOTSP, MOVRPTW	Metalearning model& attention encoder–decoder framework	2	Actor-critic	20,50,100
[39]	MOTSP	MHA,Att encoder & MHA, PtrNet decoder	1	REINFORCE	40.70.100.150,200
[94]	MOOP	CNN encoder and GRU decoder	1	REINFORCE	100,200,500,1000
[95]	MOTSP	MHA encoder & MHA, PtrNet decoder	1	REINFORCE	100,150,200
[96]	MOVRPTW	MHA encoder & MHA, PtrNet decoder	1	REINFORCE	20,50,100,150

EL = Evolutionary learning

time, eliminating the need for retraining with new problem instances. Additionally, the integration of deep learning enhances the scalability of DRL, allowing it to effectively handle problems of varying sizes. The studies by Xu et al. [69] exemplify this approach, where a model is initially trained on small problems and then transferred to handle larger problems unseen.

Autonomous optimization: Developing well-performing OR methods requires substantial domain expertise and engineering effort. In contrast, DRL algorithms operate through a data-driven, self-driven learning mechanism that only needs reward functions and previous actions. Once trained, deep learning autonomously adapts its search policies to the encountered problem distribution, eliminating the need for designing problem-specific rules. Thus, DRL algorithms exhibit promise in handling complex systems that are challenging to represent through solvable optimization models or heuristic search rules.

Online responsiveness: Most OR methods follow a create–improve process that assumes a deterministic model and aims to improve solution quality incrementally, which can perform well in scenarios with perfect and complete information. However, these methods may suffer from performance degradation when confronted with uncertain factors or dynamic events (such as scenarios where customers adjust their demand over time or new customers arrive) impacting decisions generated offline. In contract, RL and DRL approaches are trained offline but deployed in real time, enabling dynamic decision-making in sequence based on actual situations. This provides decision-makers with the maximum flexibility to react to changes and accommodate uncertainties while maintaining the high-quality solutions.

Moreover, note that the end-to-end methods are not flawless. The lack of inherent explainability, a tedious training process, and the need for hyperparameter tuning are the main drawbacks. The absence of interpretability hampers the understanding of each component's contribution to the final solution. The training process and hyperparameter tuning are time-consuming. Therefore, it is imperative to improve interpretability and optimize the training process through algorithmic advancements, parallel computing, or the use of specialized hardware accelerators. In addition, the solution quality of DRL algorithms needs further improvement.

5 RL- and DRL-based Operations Research Methods

Some studies adopt traditional OR algorithms as the foundation, using learning models to assist in exploring the solution space or guiding the search process [113–115]. We classify these approaches into three categories: RL- and DRL-based heuristics, metaheuristics, and hyperheuristics.

5.1 RL- and DRL-based Heuristics

For routing problems, several local search operators have been suggested. Suppose that we want to transform a solution from $\pi_t = (v_1, \ldots, v_i, v_{i+1}, \ldots, v_{j-1}, v_j, \ldots, v_M)$ to π_{t+1}. The node swap operation exchanges the locations of two arbitrarily chosen nodes (v_i, v_j), resulting in $\pi_{t+1} = (v_1, \ldots, v_j, v_{i+1}, \ldots, v_{j-1}, v_i, \ldots, v_M)$. The 2-optimization method (2-opt) selects 2 links (x_1, x_2) from the current tour, removes them, and reconnects the thus-generated segments with (y_1, y_2) in a distance-decreasing way. In this case, $\pi_{t+1} = (v_1, \ldots, v_j, v_{j-1}, \ldots, v_{i+1}, v_i, \ldots, v_M)$ (v_i denotes a node and x_i, y_i denote edges). It can also be expressed as a subsequence reversal between (v_i, v_j). 3-opt is an extension of 2-opt, replacing three links (x_1, x_2, x_3) with another three links (y_1, y_2, y_3) and then selecting a better tour from the two options. It is worth noting that 2-opt has only one relink manner, whereas 3-opt has seven. Figure 4 illustrates the three classical methods mentioned above.

By summarizing the relevant literature presented in Table 6, we can observe that learning the local search operators through interactions using MDP follows a similar paradigm. *State.* The state s_t is represented by either π_t or a 2-element tuple $[\pi_t, \pi_t^{best}]$, where π_t and π_t^{best} are the incumbent and the lowest distance (i.e., cost) solutions so far, respectively. And $s_0 = [\pi_0, \pi_0]$. *Action.* The action consists of two parts: one is to select the nodes to be exchanged or links to be removed, and the other is to choose a reconnecting manner, i.e., $a_t = [a_{reg}, a_{ope}]$. Considering that there is only one reconnecting type in 2-opt, a_t can be abbreviated to $[(x_1, x_2)]$, where x_1, x_2 corresponds to the two edges to be replaced. And in 3-opt, $a_t = [(x_1, x_2, x_3), a_{3-opt}]$, where x_1, x_2, x_3 denotes the three selected links, and a_{3-opt} is the reconnecting type out of a total of seven. *Reward.* Reward is the improvement in the current best found solution π_t^{best} after action a_t, i.e., $r_t = L(\pi_t) - L(\pi_t^{best})$. *Policy.* The options of a_{reg} and a_{ope} rely on different policies, both factorized by deep neural networks. The parameters are then trained using policy-based DRL approaches. For example, Kim et al. [116] use graph network and reinforcement learning to guide the selection of sub-tours for swapping in min–max mTSP and min–max VRP.

Lin–Kernighan–Helsgaun (LKH) algorithm is a commonly used baseline in VRP. Given an initial tour, LKH randomly selects a node v_1 from candidate set $\mathcal{S}$ and sets $i = 1$, followed by the alternate selection of the edges x_i to be removed and the edges y_i to be added. When iterating to node v_{2i-1}, the next step is to select the edge x_i for deletion, which essentially means to determine the node v_{2i} connecting with v_{2i-1}. Similarly, selecting the edge y_i corresponds to finding the node v_{2i+1}. The selection of v_{2i+1} is to sequentially traverse the set of candidates $\mathcal{S}$ of v_{2i} until a node is found that meets the condition $g_i = \sum_{j=1}^{i}(l(x_j) - l(y_j)) > 0$ is found

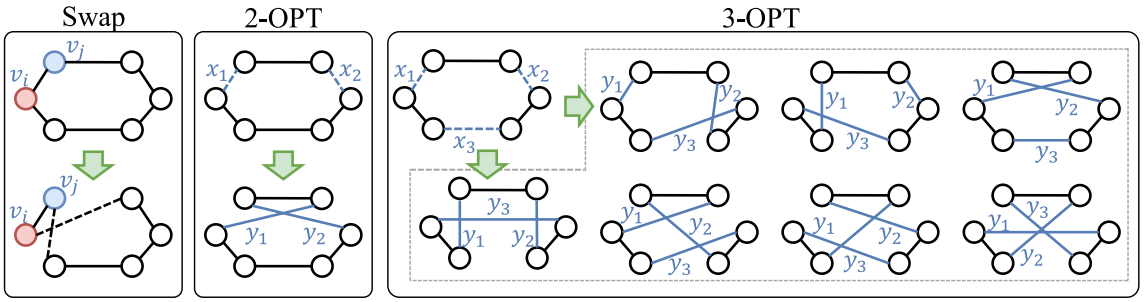


Fig. 4 Illustrations of node swap, 2-opt, and 3-opt operators

Table 6 Literature overview of the RL- and DRL-based local search operators

Reference	Problem	State	Action	Reward	Policy network	Training method
[117]	TSP	Current solution, lowest cost solution	Select a node pair and execute 2-opt	Improvement upon current best solution	GCN & RNN-based encoder and decoder	Policy gradient
[118]	TSP	Current solution, lowest cost solution	Select three node and a relink way from seven 3-opt	Improvement upon current best solution	GCN & bi-LSTM encoder, PM decoder, FiLM-Net decoder	Actor-critic
[119]	TSP,VRP	Current solution	Select a node pair and execute 2-opt	Improvement upon current best solution	Attention encoder and decoder	Actor-critic
[120]	TSP,VRP	Current solution	Select a node pair and execute 2-opt	Inverse RL without explicit rewards	Attention encoder and decoder	Soft actor-critic
[18]	MOTSP, MOVRP	Current solution	Select a node pair and execute 2-opt	Improvement upon current best solution	NN	Actor-critic

FiLM-Net = Feature-wise linear modulation network; bi-LSTM = bidirectional LSTM

(g_i is the gain of replacing $(x_1, \ldots, x_i)$ with $(y_1, \ldots, y_i)$ and $\mathrm{l}(\cdot)$ represents the length of the corresponding edge). For $i \geq 2$, if v_{2i} connects back to v_1, a new solution is constructed (see Fig. 5, left-hand side). The candidate set $\mathcal{S}$ is determined by an α-value based on the minimum spanning tree, known as the α-measure. The α-measure of an edge is the increased length on a minimum 1-tree when minimum 1-tree is required to contain this edge. The α-measure further improves the performance of the α-value by adding a π-value as penalties to each node when computing the distance between two nodes. LKH-3 is an extension of LKH that allows searching in the infeasible solution space and defines violation functions to evaluate the violation extent of the given constraints. A solution is improved by k-opt in LKH-3 when the violation function is reduced, or the violation function is unchanged while the optimization objective is reduced. A solution with zero violation values is feasible.

Zheng et al. [121] propose using three RL methods, namely Q-learning, Sarsa, and Monte Carlo, to replace the traversal operation when selecting the suitable edges to be added in the candidate set. They define a k-opt search process as an episode in RL. A state s_t corresponds to the starting point of the edge to be added, i.e., the node v_{2i} of edge y_i. An action a_t is to select the endpoint of edge y_i from the candidate set $\mathcal{S}$ of node v_{2i}. The reward represents the cost reduction achieved by replacing edge x_i with edge y_i. An adaptive Q-value is employed to replace the constant α-measure, combining node distance and α-measure, as the metric for the selection and sorting of candidate nodes. Experiments demonstrate that this algorithm significantly outperforms LKH. Xin et al. [122] introduce the NeuroLKH algorithm, using a sparse graph network to simultaneously learn edge scores and node penalties. Edge scores are trained using supervised learning to create the set of edge candidates, while node penalties are trained through unsupervised learning (UL) to transform edge distances. It reports better or similar results compared to LKH for instances with fewer than 6,000 cities. Zheng et al. [123] propose another algorithm, called VSR-LKH-3, that combines the variable strategy reinforcement learning method with LKH-3 for TSP variants. Zheng et al. [124] further introduce a reinforced hybrid algorithm that integrates the variable strategy reinforcement learning method with LKH and the edge assembly crossover genetic algorithm (EAX-GA) for very large TSP

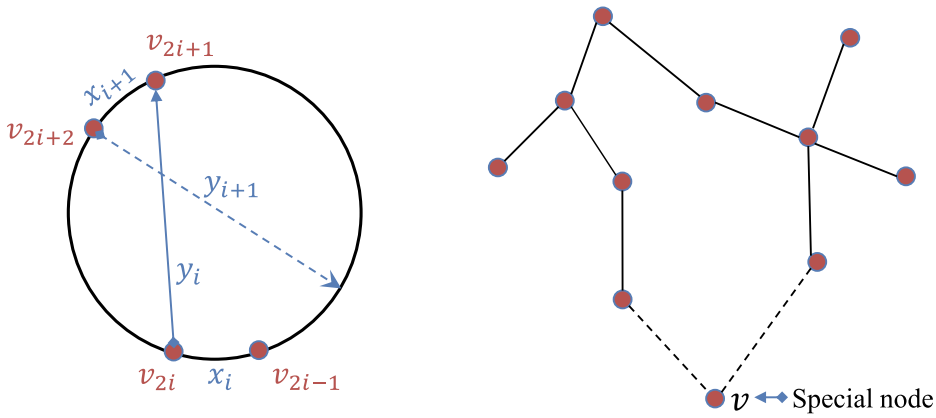


Fig. 5 Graphical representation of LKH

(a) Node choice of Lin–Kernighan algorithm (b) A 1-tree

Table 7 Literature overview of the RL- and DRL-based LKH algorithms

Reference	Underlying algorithm	Training method	Strategy
[121]	LKH	Q-learning, Sarsa and Monte Carlo	Change the method of traversing the candidate set and let RL select appropriate edges to be added in the candidate set
[123]	LKH-3		
[124]	LKH,EAX-GA		
[122]	LKH	SL,UL	Learn edge scores and node penalties

instances with up to 85,900 nodes. The summary and comparison of the aforementioned methods can be found in Table 7.

5.2 RL- and DRL-based Metaheuristics

5.2.1 Ant System and Ant Colony System

Ant system and ant colony system comprise two main phases: route construction and pheromone update. *Route construction.* Ant represent vehicles and select customers incrementally to build routes. For the ant k positioned in the customer v_i, it employs a probabilistic distribution taking into account both heuristic information and pheromone density, to determine which customer v_j to visit next. To reinforce the feedback of optimal information, the ant colony system supplements the ant system with a pseudorandom proportional rule. *Pheromone update.* Ant system updates the trail intensity as $\tau_{ij} = (1-\xi)\tau_{ij} + \xi\tau_0$ whenever ant k moves from v_i to v_j. Ant colony system incorporates two types of pheromone updates: local and global. The local update aligns with ant system, while the global update is exclusively targeted at the best ant that constructs the shortest tour, as $\tau_{ij} = (1-\rho)\tau_{ij} + \rho\Delta\tau_{ij}^{bs}, \forall(v_i, v_j) \in \pi^{bs}$, where $\Delta\tau_{ij}^{bs} = 1/L^{bs}$, ρ is the volatility coefficient of pheromone, τ_0 is the initial pheromone value, and L^{bs} is the length of best tour.

Ant-Q is the first work exploring the connection between ant system (or ant colony system) and RL, specifically Q-learning [125]. It replaces the pheromone trail with AQ-value, the counterpart of Q-value in RL. The objective of Ant-Q is to learn AQ-value for discovering approximate optimal solutions. Ant-Q employs a pseudo-Q-value function as the value function with a similar but varied learning function, as: $AQ_{ij} = (1-\rho)AQ_{ij} + \rho(\Delta AQ_{ij} + \gamma \max AQ_{js})$, where ρ and γ are the learning step and the discount factor, $\Delta AQ_{ij} = W/L^{bs}$ is the delayed reward update, $\max AQ_{js}$ is maximum Q-value that receive from external environment, k_{bs} is the agent who find the best route with length L^{bs}, W is a parameter, set to 10. Building on this foundation, Lee [126], R. et al. [127], and Kamei and Ishikawa [128] have utilized eligibility trace, distributed Q-learning, and TD(0)-learning to improve optimization robustness and expedite convergence of Ant-Q. Additionally, Dries and Peterson [129] integrate hierarchical reinforcement learning (HRL) with ant system to address TSP with large scales. They first employ data clustering techniques such as k-means and G-means to decompose the overall TSP into smaller subproblems, and then construct paths between different sub-tasks recursively. This algorithm can apply to TSP instances with up to 5000 nodes, surpassing the capabilities of AS alone.

5.2.2 Genetic Algorithm

Genetic algorithm (GA) is a widely employed evolutionary algorithm for the VRP. GA contains two fundamental operators: crossover for generating offspring and mutation for maintaining the diversity of individuals encoding solutions. The edge assembly crossover genetic algorithm (EAX-GA) [130] is an enhanced version that produces offspring solutions by combining edges from two-parent solutions and adding a few short edges to ensure the feasibility of new solutions. The biased random-key genetic algorithm (BRKGA) introduces random key to maintain feasibility of solutions from parents to offspring, and divide the population into two subpopulations: the elite subpopulation and non-elite subpopulation (refer to [131] for more details).

The performance of the genetic algorithm heavily depends on the quality of initial solutions. Therefore, de Lima Junior et al. [132] and Alipour et al. [133] employ Q-learning and multi-agent RL, respectively, to generate improved initial solutions for solving the TSP and its variants. In this case, RL starts from an empty solution and incrementally selects a node at every decision step until a complete solution is achieved. A state s_t represents the partial tour composed of the nodes already chosen, action a_t is the next node to be visited, and reward r_t denotes the reciprocal of the distance between node a_t and a_{t-1}. dos Santos et al. [134] update the Q-value based on the current best solutions generated by GA in each iteration, further enhancing the interaction between GA and RL. Mutation is widely acknowledged as a crucial element in GA. Thus, Liu and Zeng [135] improve EAX-GA by incorporating Q-learning as the mutation operator. In this scenario, RL initiates with a complete solution and perturbs the local sequence to achieve the mutation. State s_t corresponds to the current node v_i, action a_t is to select the next node v_j with a shorter length l_{ij} than $l_{i(i+1)}$, and reward r_{ij} is $1/l_{ij}$. The predefined constant hyperparameters in GA may not be ideal, as their performance vary based on the search stage and tested instance. Therefore, Chaves et al. [131] and Schenekemberg et al. [136] employ the Q-learning with exponential decay ϵ-greedy policy to use feedback information to dynamically choose the parameters of BRKGA, including population size, percentage of elite individuals, percentage of mutant individuals, and the probability of inheriting a key from an elite parent. Experiments demonstrate that this algorithm can find optimal solutions or solutions close to optimal within a competitive computational time for the given instances. Additionally, Qi et al. [137] utilize Q-learning to guide the selection of local operators, such as 2-opt, which operate on solutions after crossover to further enhance their quality.

5.2.3 Neighborhood Search

Large neighborhood search (LNS) is a metaheuristic that explores the space of all possible solutions by iteratively applying destroy and repair operators to a starting solution. Let π is the current route. The destroy operator removes parts of the solution, creating an infeasible solution. It often involves stochastic elements to vary the deleted portions. The repair operator then corrects the infeasible solution by completing incomplete tours or assigning unassigned customers to existing tours, thus creating a feasible solution π'. Variable neighborhood search (VNS) is a local search-based algorithm that systematically changes neighborhood structures within the search procedure to extensively explore the search space and escape from local optima.

The performance of LNS relies heavily on the quality of the destroy and repair operators. Therefore, Chen et al. [138], Gao et al. [139] and Kong et al. [140] use actor-critic or PPO as adaptive destroy operators to identify removal candidates. They then employ a predefined least cost repair operator to restore connections by inserting removed nodes in a feasible order with minimum detour. Hottung and Tierney [141] implement two simple destroy procedures, namely point-based destroy and tour-based destroy, to disrupt current solutions, and then train REINFORCE to reconnect the incomplete parts. Wang et al. [142] employ DQN to select a combination of destroy and repair operators, along with a charging strategy. The selection of the appropriate neighborhood structure at specific points in the search or after improvements is vital for achieving global optimization performance in VNS. Therefore, Queiroz dos Santos et al. [145], Lopes Silva et al. [146] employ Q-learning to select neighborhood operators. In this context, the state is the current neighborhood operator, an action is defined as the transition from one operator to another, and the reward is based on the fitness function value of the solution obtained with the application of the current neighborhood operator. The summary of the above methods can be found in Table 8.

5.3 RL- and DRL-based Hyperheuristics

Hyperheuristic is a search method that selects or generates new heuristics from a set of potential heuristics instead of directly solving problems, often described as 'heuristics to choose heuristics.' It primarily comprises two key components: a high-level heuristic (HLH, includes a heuristic selection mechanism and acceptance criterion), a pool of low-level heuristics (LLHs). If the heuristics pool includes metaheuristic algorithms, these algorithms come with their own acceptance criteria. Hyperheuristics can leverage the advantages of various heuristics or metaheuristics by selecting the most suitable selection–acceptance pair under specific

Table 8 Literature overview of the RL- and DRL-based metaheuristics

Underlying algorithm	Description	Reference	Problem	RL method
Ant colony system	RL replaces the pheromone trails with Q-value	[125]	TSP	Q-learning
		[126]	TSP	TD(λ)
		[129]	TSP	HRL
		[127]	VRPTW	Distributed Q-learning
		[128]	TSP	TD(0)
Genetic algorithm	RL is employed to generate an initial solution or improve existing solutions	[132]	TSP	Q-learning
		[133]	TSP	Multi-agent RL
		[137]	TSP	Q-learning
	RL is used as the mutation operator	[135]	TSP	Q-learning
	RL is used to configure the parameters of GA	[136]	DARP-PFCC	Q-learning
		[131]	FTSP	Q-learning
Neighborhood search	RL is used as the destruction operator in LNS	[138]	VRP	Actor-critic
		[139]	VRP	PPO
		[140]	VRPPD	PPO
	RL is used as the repair operator in LNS	[141]	VRP,SDVRP	REINFORCE
	RL is used to select the repair and destruction operator	[142]	EVRP	DQN
		[143]	VRP, HVRP, VRPTW	DQN
		[144]	VRP	PPO
	RL is used to select neighborhood operators in VNS	[145]	TSP	Q-learning
		[146]	VRP	Q-learning
		[147]	VRP	Multi-arm Bandit
		[148]	VRP, MDVRP	Multi-agent RL

DARP-PFCC = Dial-a-ride problem with private fleet and common carrier

circumstances. As such, the learning-based components are crucial elements of these hyperheuristic methods.

Most learning-based hyperheuristics utilize RL and DRL as heuristic selection mechanism to determine the most appropriate heuristic for the next iteration, as summarized in Table 9. Hence, the action space is the pool of LLHs. Sabar et al. [149], Soria-Alcaraz et al. [150], and Rodríguez-Esparza et al. [151] use the multi-armed bandit to select the problem-specific heuristics. Yao et al. [152] develop a Q-learning-based multi-objective hyperheuristic for route planning that considers both safety and distance in a large-scale smart city. Its reward is defined as the difference in the non-dominated front rank of the solutions before and after the action, specifically tailored for multi-objective optimization settings. Chen and Tian [153] propose a NeuRewriter model that concurrently learns a region-picking policy to identify nodes requiring rescheduling and a rule-picking policy to select heuristics, such as swap, for rewriting the current solution. The authors utilize an LSTM encoder and train region-picking and rule-picking policies jointly by employing the Q-actor-critic algorithm. Lu et al. [154] propose an iterative improvement method named 'Learn to Improve (L2I).' The model begins with a randomly generated initial solution and employs REINFORCE to select two categories of predefined low-level heuristics: improvement operators and perturbation operators. And then, the heuristics are used to destroy the existing solution and create a new one. Similarly, in [155] and [156], the low-level algorithm pool is also predefined with heuristic rules. Qin et al. [158] propose an RL-based hyperheuristic that employs several well-known metaheuristics, including artificial bee colony (ABC), ant colony optimization (ACO), cuckoo search (CS), GA, particle swarm optimization (PSO), and simulated annealing (SA), as low-level heuristics, with distributed proximal policy optimization (DPPO) serving as the high-level selection strategy. DPPO is employed to calculate the policy gradient, determining the next algorithmic to be taken. After that, the selected metaheuristic runs for a fixed number of generations,

and a new population is generated. Accordingly, the reward is defined as the difference in score between the final state and the initial state to assess the correctness of the LLH selection sequence. Choong et al. [159] and Shang et al. [160] argue that in this design mode, the acceptance criteria of the solution have an impact on the result of the solution comparable to that of the LLH. Thus, Q-learning is used to guide the selection of 30 evolution–acceptance pairs, comprising six LLH selection methods and five move acceptance methods.

5.4 General Observations and Insights

In addition, a few papers also combine RL with constraint programming [161] and memetic algorithms [162]. RL- and DRL-based OR methods utilize classical algorithms as their core, belong to the category of improvement heuristics. In other words, these methods continuously enhance the quality of the solution by refining a given initial solution. Through a literature review, we observe that RL and DRL techniques are integrated into operations research methods mainly to improve solution quality and accelerate convergence rates for various purposes. This enhancement can be derived from several key aspects. (i) Initialization: OR methods typically begin from an initial solution or a population of solutions, and then continually improve solution quality by refining the given initial solutions. Thus, the performance of these algorithms heavily depends on the quality of the initial solutions. RL or DRL techniques can contribute to generating high-quality initial solutions by leveraging the knowledge from good solutions on similar instances. In addition, due to the NP-hardness of the VRP, obtaining an initial feasible solution might be challenging and time-consuming, especially for more constrained VRP variants. End-to-end DRL mechanisms can promptly provide effective initial solutions after training, expediting the overall problem-solving process. (ii) Algorithm selection: There are various heuristics and metaheuristics suitable for the VRP, but determining which one is likely to perform best for a given instance remains a challenging problem. RL and DRL techniques can dynamically select the most suitable operator under specific circumstances according to the evolution trajectories, the sequence of search operators, and operator performance, thereby enhancing the solution quality of OR methods. In addition, RL and DRL algorithms can replace the tedious traverse operation in the selection process, reducing the amount of calculation.

(iii) Evaluation: Most OR methods employ a destroy-and-repair strategy to enhance their performance. RL and DRL techniques can interact with the models to autonomously assess the removal of nodes and the reconnection of generated fragments, selecting the most effective methods for destroying and repairing. (iv) Parameter setting: OR methods require manual configuration of a set of parameters before the search process begins. Predefined constant hyperparameters may not be ideal, as their performance varies based on the search stage and tested instances. RL and DRL techniques can enable adaptive parameter control, utilizing feedback to adjust parameter levels during the search process. This adaptive approach reduces reliance on expert knowledge and enhances the robustness and flexibility of OR methods. (v) Guiding search: Compared to OR methods, RL and DRL can extract valuable knowledge from the generated data throughout the search process and use this information to guide the algorithm, preventing it from getting trapped in local optima and directing it toward more promising directions, thereby enhancing the solution quality and decreasing calculation time.

6 Experimental Study

This section compares representative algorithms from two major categories, end-to-end RL and DRL methods, as well as RL and DRL-based OR methods. The evaluation is based on three metrics: distance, optimality gap, and computational time, using CVRP instances uniformly generated within the unit square.

6.1 Experimental Setting

For end-to-end RL and DRL methods, as described in Sect. 4, the attention-based encoder–decoder architecture (AM) proposed by Kool et al. [30] has become a foundational framework in this field. Subsequently, most methods have built upon this architecture, enhancing algorithm performance and adapting to various variants by modifying the network structure or training strategies. Therefore, AM was selected as a representative algorithm for evaluation. Additionally, POMO [53] is included, which enhances the attention mechanism by parallel sampling of multiple solution trajectories. For RL- and DRL-based OR methods, L2I [154] and NLNS [141] are widely cited and often adopted as baselines in subsequent studies. Accordingly, these three algorithms are selected as representative methods for evaluation. And the hyperparameters of these algorithms are shown in Table 10.

To ensure a comprehensive and fair comparison, the selected RL and DRL algorithms are benchmarked against widely used baseline methods in the CVRP domain, including the classical exact solver Gurobi, the well-established heuristic LKH-3, and popular metaheuristic approaches such as GA, ACO, ALNS, and VNS. The computation time for Gurobi is limited to 600 s. The hyperparameters used for the metaheuristic algorithms are listed in Table 11.

Following the methodology of Bello et al., CVRP datasets are generated with instances containing 20, 50, and 100 customers. The coordinates of both customers and the depot are

Table 9 Literature overview of the RL- and DRL-based hyperheuristics

Strategy	Reference	Problem	HLH		LLH
			Selection method	Acceptance	
Use RL as heuristic selection mechanism	[149]	DVRP	Multi-armed bandit	Gene expression programming	9 in total, i.e., move, swap
	[150]	VRPTW	Multi-armed bandit	Credit assignment	8 in total, i.e., shift, 2-opt
	[152]	MOVRP	Q-learning	-	m-hop path to n-hop path
	[153]	VRP	Q-actor-critic	-	19 categories, i.e., swap
	[154]	VRP	REINFORCE	-	2-Opt,relocate,cross, symmetric exchange
	[158]	HVRP	DPPO	-	ABC,ACO,CS,GA,PSO,SA
	[155]	VRP	PPO	Annealing approach	Removal and insert operators
	[156]	VRPTW	Multi-armed bandit	Annealing approach	Improvement,removal, insertion heuristics
	[151]	CEVRP	Multi-armed bandit	Annealing approach	8 in total, i.e., swap, 2-opt
	[157]	TSP,VRP	PPO	-	Relocate, exchange, 2-opt
Use RL to determine selection–acceptance pair	[159]	TSP,VRP	Q-learning (Improvement, accepted, improvement over time, speed, speed accepted, speed new)	OI,AM,MA, SA,LA,NA	Mutation or ruin–recreate heuristics
	[160]	MCVRP		FA,OI,MA, SA	Swap, insertion,reversion, sequence swap,aggregation, sequence insertion

DARP-PFCC = Dial-a-ride problem with private fleet and common carrier; OI = only improvement; AM = all moves; MA = Metropolis acceptance; SA = simulated annealing; LA = late acceptance; NA = naive acceptance; FA = fully acceptance

Table 10 Hyperparameters used for the RL and DRL algorithms

Parameters	AM	POMO	L2I	NLNS
Training epochs	100	100	4,000	100
Training samples	1, 280, 000	640,000	2000	25,000
Batch size	512	64	1,000	256
Learning rate	1e-4	1e-4	1e-3	1e-4

Table 11 Hyperparameters used for the metaheuristic algorithms

Algorithm	Hyperparameters
GA	iterations 1000, population size 100, crossover rate 0.9, mutation rate 0.2, tournament size 10
ACO	iterations 1000, ants 100, iterations 1000, pheromone influence 1.0
	heuristic influence 5.0, evaporation rate 0.5, pheromone update constant 100
VNS	iterations 1000, maximum shaking strength 10
ALNS	iterations 1000, removal ratio 0.2, frequency of weight updates 50, reaction factor 0.08

uniformly sampled from the unit square $[0, 1] \times [0, 1]$. Vehicle capacities are set to 30, 40, and 50 for the 20-, 50-, and 100-customer instances, respectively. Customer demands are randomly drawn from the discrete set $\{1, \ldots, 9\}$, while the depot demand is fixed at zero. For each problem size, a fixed set of 100 test instances is generated to serve as a common benchmark for all algorithms. RL and DRL algorithms are trained on independently and randomly generated training datasets, ensuring a fair and consistent evaluation across methods on the standardized test set. All algorithms are compared on the CVRP test set using three key metrics: solution distance, optimality gap, and computational time. Solutions obtained by Gurobi serve as the reference for optimal solution distance, against which the results of other methods are compared to compute the optimality gap. All algorithms are executed on a computer equipped with an Intel Core i9-14900K CPU @ 5.20 GHz and 64 GB of RAM.

6.2 Results and Discussion

Table 12 presents a comparison of solution distances and computation times for various algorithms across CVRP instances of different sizes (20, 50, and 100 customers). For each algorithm, the average route length, optimality gap, and runtime are reported.

Overall, among RL- and DRL-based methods, L2I delivers the best solution quality, outperforming both metaheuristic algorithms and end-to-end RL and DRL approaches on CVRP20, CVRP50, and CVRP100. Notably, it even surpasses LKH-3 on CVRP100. However, these algorithms tend to require longer computation times; for instance, L2I takes over 30 s to solve CVRP100. This is primarily because they rely on heuristic search methods to identify optimal solutions, which involves repeated cycles of solution generation and evaluation.

Among end-to-end RL and DRL methods, AM (sampling) outperforms most metaheuristic algorithms on CVRP20 and CVRP50, ranking just behind VNS, while POMO shows similar performance on CVRP20. On CVRP100, both AM (sampling) and POMO surpass all metaheuristic algorithms, demonstrating their potential for solving large-scale problems. Furthermore, the performance gap between AM (sampling) and AM (greedy) highlights the significant impact of search strategies on algorithm optimization. Notably, AM and POMO require shorter computation times than other RL- and DRL-based methods, underscoring their overall efficiency advantage.

7 Conclusions

In recent years, RL and DRL techniques have been extensively applied to solve the VRP and its variants. This paper offers a comprehensive and technical review of the end-to-end RL and DRL framework and the integration of RL and DRL techniques in various operations research methods. In comparison, the former exhibits superior generalization to diverse problem instances, greater robustness to dynamic and uncertain scenarios, and reduced computational time. In contrast, the latter consistently yields solutions of higher quality. Throughout the paper, a set of common challenges arises in the application of RL and DRL techniques to the VRP, as follows.

First, scaling to larger problem instances remains challenging when training RL and DRL techniques on instances beyond a certain size. Attempting to utilize larger instances for training poses significant time and computational hurdles, especially when applying sophisticated RL and DRL techniques to complex optimization problems. Second, current RL and DRL methods are primarily applied to relatively simple VRP scenarios and their variants. As the problem complexity increases, achieving high-quality solutions typically requires more complex models, which in turn raises the demands of model training. Third, RL and DRL training requires updating the weights of neural networks through numerous iterations over extensive datasets, demanding substantial computational power. Effectively addressing this challenge requires exploring advanced algorithms, leveraging parallel computing, or incorporating hardware accelerators. Fourth, most RL and DRL algorithms employ a random uniform distribution to generate data and train the policy network. However, real-world situations pose addi-

Table 12 Comparison of solution distance and computation time for different algorithms

Categories	Algorithms	CVRP20			CVRP50			CVRP100		
		Distance	Opt. Gap	Time (s)	Distance	Opt. Gap	Time (s)	Distance	Opt. Gap	Time (s)
Classic methods	Gurobi	7.73	0.00%	600	12.67	0.00%	600	20.70	0.00%	600
	LKH-3	7.78	0.65%	39.19	12.79	0.95%	112.71	21.28	2.80%	309.83
	ALNS	8.36	8.15%	1.05	14.55	14.84%	2.31	22.73	9.81%	3.43
	ACO	8.41	8.80%	16.24	14.35	13.26%	72.45	24.47	18.21%	235.68
	VNS	7.94	2.72%	2.85	13.16	3.87%	34.99	22.10	6.76%	169.42
	GA	8.39	8.54%	1.88	14.80	16.81%	3.42	25.24	21.93%	6.84
End-to-end RL and DRL	AM(greedy)	8.38	8.41%	0.58	14.18	11.92%	1.35	22.87	10.48%	2.03
	AM(sampling)	7.98	3.23%	0.75	13.25	4.58%	1.67	21.86	5.60%	2.41
	POMO	7.96	2.59%	0.92	13.11	3.47%	1.97	21.43	3.53%	2.64
RL- and DRL-based	L2I	7.88	1.94%	8.89	13.07	3.16%	18.34	21.23	2.56%	39.18
	NLNS	8.04	4.01%	7.54	13.66	7.81%	13.53	22.09	6.71%	25.52

tional complexities compared to simulated data. Challenges arise when node locations deviate from specific distributions or when these distributions are unknown. Furthermore, the utilization of simulated data introduces difficulties in reproducibility.

A set of future research directions for the application of RL and DRL techniques to solve the VRP is elaborated as follows.

- A common critique of ML models is their inherent lack of interpretability. Addressing this concern is pivotal for extending the applicability of RL and DRL techniques. A promising approach involves developing theoretical research that harnesses methods from various disciplines, including mathematics, computer science, and OR.
- The use of emerging deep representation algorithms, such as generative adversarial network (GAN) and BERT, along with advanced training methods (like generative adversarial imitation learning and transfer learning) to empower existing RL and DRL paradigms is a long-standing promising direction. For example, GAN can generate realistic and diverse routing solutions, BERT-style architectures effectively capture sequential dependencies in customer visits, and transfer learning enables knowledge reuse across VRP variants with different constraints or scales, potentially allowing models trained on simpler VRP scenarios to be applied to more complex ones.
- Integrating RL and DRL techniques with exact methods or solution solvers. In the literature, it has been observed that RL and DRL techniques have primarily been combined with approximation algorithms. If RL and DRL techniques are employed to accelerate the optimization process of exact algorithms, the resulting solutions are expected to exhibit higher quality and interpretability.
- Numerous RL and DRL algorithms have been applied to VRP and its variants using different simulated data, hyperparameters, and benchmarks. However, a challenge is the lack of common platforms for their evaluation. The absence of standardized platforms hinders fair and meaningful comparisons. Establishing a common platform could rectify this, providing standardized inputs and instances and facilitating more equitable and reasonable comparisons.

Acknowledgements This work was supported by the Laboratory Project of Higher Education Institutions in Shandong Province–Energy System Intelligent Management and Policy Simulation Laboratory at China University of Petroleum, and Youth Innovation Team of Higher Education Institutions in Shandong Province–Data Intelligence Innovation Team at China University of Petroleum. We sincerely appreciate the valuable feedback and insightful comments from the Editor-in-Chief,

Associate Editor, and reviewers. Their suggestions have been instrumental in improving the quality of our manuscript.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Adamo, T.; Gendreau, M.; Ghiani, G.; Guerriero, E.: A review of recent advances in time-dependent vehicle routing. Eur. J. Oper. Res. **319**(1), 1–15 (2024)
2. Hildebrandt, F.D.; Thomas, B.W.; Ulmer, M.W.: Opportunities for reinforcement learning in stochastic dynamic vehicle routing. Comput. Oper. Res. **150**, 106071 (2023)
3. Chen, L.; Zhu, Y.; Lee, K. Y.: Two-step genetic algorithm for dynamic route optimization of electric vehicles based on demand analysis. *Arab. J. Sci. Eng.* (2025)
4. Zhao, L.-h.; Zhang, J.; Bah, M.J.; Li, Z.; Ying, J. J.-C.; Muthanna, A.; Elgendy, I.A.: Energy-efficient task offloading in multi-server mobile edge computing networks: a deep reinforcement learning approach. *Arab. J. Sci. Eng.*. (2025)
5. Bai, R.; Chen, X.; Chen, Z.-L.; Cui, T.; Gong, S.; He, W.; Jiang, X.; Jin, H.; Jin, J.; Kendall, G.; Li, J.; Lu, Z.; Ren, J.; Weng, P.; Xue, N.; Zhang, H.: Analytics and machine learning in vehicle routing research. Int. J. Prod. Res. **61**(1), 4–30 (2023)
6. Bogyrbayeva, A.; Meraliyev, M.; Mustakhov, T.; Dauletbayev, B.: Machine Learning to Solve Vehicle Routing Problems: A Survey. IEEE Trans. Intell. Transp. Syst. **25**(6), 4754–4772 (2024)
7. Mazyavkina, N.; Sviridov, S.; Ivanov, S.; Burnaev, E.: Reinforcement learning for combinatorial optimization: A survey. Comput. Oper. Res. **134**, 105400 (2021)
8. Yan, Y.; Chow, A.H.F.; Ho, C.P.; Kuo, Y.-H.; Wu, Q.; Ying, C.: Reinforcement learning for logistics and supply chain management: Methodologies, state of the art, and future opportunities. Trans. Res. Part E: Logist. Trans. Rev. **162**, 102712 (2022)
9. Qin, Z.T.; Zhu, H.; Ye, J.: Reinforcement learning for ridesharing: An extended survey. Trans. Res. Part C: Emerg. Technol. **144**, 103852 (2022)
10. Wang, Q.; Tang, C.: Deep reinforcement learning for transportation network combinatorial optimization: A survey. Knowl.-Based Syst. **233**, 107526 (2021)
11. Raza, S.M.; Sajid, M.; Singh, J.: Vehicle routing problem using reinforcement learning: recent advancements. In *Adv. Mach. Intell. Signal Process.* 269–280. (2022)
12. Qiu, H.; Wang, S.; Yin, Y.; Wang, D.; Wang, Y.: A deep reinforcement learning-based approach for the home delivery and installation routing problem. Int. J. Prod. Econ. **244**, 108362 (2022)
13. Gama, R.; Fernandes, L.H.: A reinforcement learning approach to the orienteering problem with time windows. Comput. Oper. Res. **133**, 105357 (2021)
14. Karimi-Mamaghan, M.; Mohammadi, M.; Meyer, P.; Karimi-Mamaghan, A.M.; Talbi, E.-G.: Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art. Eur. J. Oper. Res. **296**(2), 393–422 (2022)
15. Dantzig, G.B.; Ramser, J.H.: The Truck Dispatching Problem. Manage. Sci. **6**(1), 80–91 (1959)
16. Bello, I.; Pham, H.; Le, Q.; Norouzi, M.; Bengio, S.: Neural combinatorial optimization with reinforcement learning. In *International Conference on Learning Representations*. (2017)
17. Zhao, J.; Mao, M.; Zhao, X.; Zou, J.: A hybrid of deep reinforcement learning and local search for the vehicle routing problems. IEEE Trans. Intell. Transp. Syst. **22**(11), 7208–7218 (2021)
18. Zheng, Z.; Yao, S.; Li, G.; Han, L.; Wang, Z.: Pareto improver: learning improvement heuristics for multi-objective route planning. *IEEE Tran. Intell. Trans. Syst.* 1–11. (2023c)
19. Alqahtani, M.; Scott, M.J.; Hu, M.: Dynamic energy scheduling and routing of a large fleet of electric vehicles using multi-agent reinforcement learning. Comput. Ind. Eng. **169**, 108180 (2022)
20. Sutton, R.S.; McAllester, D.; Singh, S.; Mansour, Y.: Policy gradient methods for reinforcement learning with function approximation. In *Proceedings of the 12th International Conference on Neural Information Processing Systems*, 1057–1063. (1999)
21. Williams, R.J.: Simple statistical gradient-following algorithms for connectionist reinforcement learning. Mach. Learn. **8**(3), 229–256 (1992)
22. Mnih, V.; Badia, A.P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; Kavukcuoglu, K.: Asynchronous methods for deep reinforcement learning. In *Proceedings of The 33rd International Conference on Machine Learning*, pp 1928–1937. (2016)
23. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O.: Proximal Policy Optimization Algorithms. (2017)
24. Watkins, C.J.C.H.; Dayan, P.: Q-learning. Mach. Learn. **8**(3), 279–292 (1992)
25. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; Hassabis, D.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)
26. Hasselt, H.v.; Guez, A.; Silver, D.: Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1). (2016)
27. Hasselt, H.: Double Q-learning. *Adv.Neural Inf. Process. Syst.***23**. (2010)
28. Wang, Z.; Schaul, T.; Hessel, M.; Hasselt, H.; Lanctot, M.; Freitas, N.: Dueling network architectures for deep reinforcement Learning. In *Proceedings of The 33rd International Conference on Machine Learning*, pp. 1995–2003. (2016)
29. Zhang, K.; He, F.; Zhang, Z.; Lin, X.; Li, M.: Multi-vehicle routing problems with soft time windows: A multi-agent reinforcement learning approach. Trans. Res. Part C: Emerg. Technol. **121**, 102861 (2020)
30. Kool, W.; Hoof, H.v.; Welling, M.: Attention, learn to solve routing problems! In *International Conference on Learning Representations*. (2018)
31. Xin, L.; Song, W.; Cao, Z.; Zhang, J.: Multi-decoder attention model with embedding glimpse for solving vehicle routing problems. Proc. AAAI Confer. Artific. Intell. **35**(13), 12042–12049 (2021)
32. Sutskever, I.; Vinyals, O.; Le, Q.V.: Sequence to sequence learning with neural networks. *Adv. Neural Inf. Process. Syst.***27** (2014)
33. Deudon, M.; Cournut, P.; Lacoste, A.; Adulyasak, Y.; Rousseau, L.-M.: Learning heuristics for the TSP by policy gradient. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pp. 170–181. (2018)
34. Lin, X.; Yang, Z.; Zhang, Q.: Pareto set learning for neural multiobjective combinatorial optimization (2022b)
35. Ye, T.; Zhang, Z.; Chen, J.; Wang, J.: Weight-specific-decoder attention model to solve multiobjective combinatorial optimization problems. In *2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pp. 2839–2844. (2022)
36. Zhu, T.; Shi, X.; Xu, X.; Cao, J.: An accelerated end-to-end method for solving routing problems. Neural Netw. **164**, 535–545 (2023)

37. Nazari, M.; Oroojlooy, A.; Takáč, M.; Snyder, L.V.: Reinforcement learning for solving the vehicle routing problem. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, pp 9861–9871. (2018)
38. Xu, Y.; Fang, M.; Chen, L.; Xu, G.; Du, Y.; Zhang, C.: Reinforcement learning with multiple relational attention for solving vehicle routing problems. IEEE Trans. Cybern. **52**(10), 11107–11120 (2022)
39. Li, S.; Wang, F.; He, Q.; Wang, X.: Deep reinforcement learning for multi-objective combinatorial optimization: A case study on multi-objective traveling salesman problem. Swarm Evol. Comput. **83**, 101398 (2023)
40. Arishi, A.; Krishnan, K.: A multi-agent deep reinforcement learning approach for solving the multi-depot vehicle routing problem. J. Manage. Anal. **10**(3), 493–515 (2023)
41. Tang, M.; Zhuang, W.; Li, B.; Liu, H.; Song, Z.; Yin, G.: Energy-optimal routing for electric vehicles using deep reinforcement learning with transformer. Appl. Energy **350**, 121711 (2023)
42. Lin, B.; Ghaddar, B.; Nathwani, J.: Deep reinforcement learning for the electric vehicle routing problem with time windows. IEEE Trans. Intell. Transp. Syst. **23**(8), 11528–11538 (2022)
43. Zhang, Z.; Liu, H.; Zhou, M.; Wang, J.: Solving dynamic traveling salesman problems with deep reinforcement learning. IEEE Trans. Neural Netw. Learn. Syst. **34**(4), 2119–2132 (2023)
44. Li, J.; Ma, Y.; Gao, R.; Cao, Z.; Lim, A.; Song, W.; Zhang, J.: Deep reinforcement learning for solving the heterogeneous capacitated vehicle routing problem. IEEE Trans. Cybern. **52**(12), 13572–13585 (2022)
45. Li, J.; Xin, L.; Cao, Z.; Lim, A.; Song, W.; Zhang, J.: Heterogeneous attentions for solving pickup and delivery problem via deep reinforcement learning. IEEE Trans. Intell. Transp. Syst. **23**(3), 2306–2315 (2022)
46. Xin, L.; Song, W.; Cao, Z.; Zhang, J.: Step-wise deep learning models for solving routing problems. IEEE Trans. Industr. Inf. **17**(7), 4861–4871 (2021)
47. Vinyals, O.; Fortunato, M.; Jaitly, N.: Pointer Networks. In *Advances in Neural Information Processing Systems*, vol. 28. (2015)
48. Sheng, Y.; Ma, H.; Xia, W.: A pointer neural network for the vehicle routing problem with task priority and limited resources. Inf. Technol. Control **49**(2), 237–248 (2020)
49. Iklassov, Z.; Sobirov, I.; Solozabal, R.; Takáč, M.: Reinforcement learning for solving stochastic vehicle routing problem. In *Proceedings of the 15th Asian Conference on Machine Learning*, pp. 502–517. (2024)
50. Li, H.; Li, G.: Learning to solve capacitated arc routing problems by policy gradient. In *2019 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1291–1298. (2019)
51. Wang, Q.; Lai, K.H.; Tang, C.: Solving combinatorial optimization problems over graphs with BERT-based deep reinforcement learning. Inf. Sci. **619**, 930–946 (2023)
52. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Polosukhin, I.: Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pp 6000–6010. (2017)
53. Kwon, Y.-D.; View Profile, Choo, J.; View Profile, Kim, B.; View Profile, Yoon, I.; View Profile, Gwon, Y.; View Profile, Min, S.; View Profile.: Pomo: policy optimization with multiple optima for reinforcement learning. *Proceedings of the 34th International Conference on Neural Information Processing Systems*, pp. 21188–21198. (2020)
54. Kim, M.; Park, J.; Kim, J.: Learning collaborative policies to solve NP-hard routing problems. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, pp. 10418–10430. (2021)
55. Zong, Z.; Wang, H.; Wang, J.; Zheng, M.; Li, Y.: RBG: Hierarchically solving large-scale routing problems in logistic systems via reinforcement learning. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 4648–4658. (2022a)
56. Bono, G.; Dibangoye, J.S.; Simonin, O.; Matignon, L.; Pereyron, F.: Solving multi-agent routing problems using deep attention mechanisms. IEEE Trans. Intell. Transp. Syst. **22**(12), 7804–7813 (2021)
57. Ren, L.; Fan, X.; Cui, J.; Shen, Z.; Lv, Y.; Xiong, G.: A multi-agent reinforcement learning method with route recorders for vehicle routing in supply chain management. IEEE Trans. Intell. Transp. Syst. **23**(9), 16410–16420 (2022)
58. Zong, Z.; Tong, X.; Zheng, M.; Li, Y.: Reinforcement learning for solving multiple vehicle routing problem with time window. ACM Trans. Intell. Syst. Technol. **15**(2), 321–3219 (2024)
59. Peng, B.; Wang, J.; Zhang, Z.: A deep reinforcement learning algorithm using dynamic attention model for vehicle routing problems. In *Artific. Intell. Algorithm. Appl.*, pp. 636–650. (2020)
60. Falkner, J.K.; Schmidt-Thieme, L.: Learning to solve vehicle routing problems with time windows through joint attention. (2020)
61. Ma, Y.; Li, J.; Cao, Z.; Song, W.; Zhang, L.; Chen, Z.; Tang, J.: Learning to iteratively solve routing problems with dual-aspect collaborative transformer. Adv. Neural. Inf. Process. Syst. **34**, 11096–11107 (2021)
62. Zou, Y.; Wu, H.; Yin, Y.; Dhamotharan, L.; Chen, D.; Tiwari, A.K.: An improved transformer model with multi-head attention and attention to attention for low-carbon multi-depot vehicle routing problem. *Ann. Oper. Res*. (2022)
63. Li, K.; Zhang, T.; Wang, R.; Wang, Y.; Han, Y.; Wang, L.: Deep reinforcement learning for combinatorial optimization: covering salesman problems. IEEE Trans. Cybern. **52**(12), 13142–13155 (2022)
64. Lei, K.; Guo, P.; Wang, Y.; Wu, X.; Zhao, W.: Solve routing problems with a residual edge-graph attention neural network. Neurocomputing **508**, 79–98 (2022)
65. Zong, Z.; Zheng, M.; Li, Y.; Jin, D.: MAPDP: cooperative multi-agent reinforcement learning to solve pickup and delivery problems. In Proceedings of the AAAI Conference on Artificial Intelligence **36**, 9980–9988 (2022)
66. Zhang, K.; Lin, X.; Li, M.: Transformer-based reinforcement learning for pickup and delivery problems with late penalties. IEEE Trans. Intell. Transp. Syst. **23**(12), 24649–24661 (2022)
67. Li, J.; Ma, Y.; Cao, Z.; Wu, Y.; Song, W.; Zhang, J.; Chee, Y.M.: Learning feature embedding refiner for solving vehicle routing problems. *IEEE Trans. Neural Netw. Learn. Syst.* (2023a)
68. Gao, H.; Zhou, X.; Xu, X.; Lan, Y.; Xiao, Y.: AMARL: An attention-based multiagent reinforcement learning approach to the min-max multiple traveling salesmen problem. *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–15. (2023)
69. Xu, Y.; Fang, M.; Chen, L.; Du, Y.; Xu, G.; Zhang, C.: Shared dynamics learning for large-scale traveling salesman problem. Adv. Eng. Inform. **56**, 102005 (2023)
70. Yuan, Z.; Li, G.; Wang, Z.; Sun, J.; Cheng, R.: RL-CSL: a combinatorial optimization method using reinforcement learning and contrastive self-supervised learning. IEEE Trans. Emerg. Top. Comput. Intell. **7**(4), 1010–1024 (2023)
71. Yang, H.; Zhao, M.; Yuan, L.; Yu, Y.; Li, Z.; Gu, M.: Memory-efficient transformer-based network model for traveling salesman problem. Neural Netw. **161**, 589–597 (2023)
72. Jiang, Y.; Cao, Z.; Wu, Y.; Song, W.; Zhang, J.: Ensemble-based deep reinforcement learning for vehicle routing problems under distribution shift. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, pp. 53112–53125. (2024)

73. Wang, Y.; Hong, X.; Wang, Y.; Zhao, J.; Sun, G.; Qin, B.: Token-based deep reinforcement learning for Heterogeneous VRP with service time constraints. Knowl.-Based Syst. **300**, 112173 (2024)
74. Wang, C.; Cao, Z.; Wu, Y.; Teng, L.; Wu, G.: Deep reinforcement learning for solving vehicle routing problems with backhauls. *IEEE Trans. Neural Netw. Learn. Syst.* pp. 1–15. (2024b)
75. Wang, M.; Wei, Y.; Huang, X.; Gao, S.: An end-to-end deep reinforcement learning framework for electric vehicle routing problem. IEEE Internet Things J. **11**(20), 33671–33682 (2024)
76. Guan, Q.; Cao, H.; Tan, J.; Jia, L.; Yan, D.; Chen, B.: Token-specific deep reinforcement learning for energy-efficient capacitated electric vehicle routing problems. Appl. Energy **396**, 126314 (2025)
77. Guan, Q.; Cao, H.; Jia, L.; Yan, D.; Chen, B.: Synergetic attention-driven transformer: A deep reinforcement learning approach for vehicle routing problems. Expert Syst. Appl. **274**, 126961 (2025)
78. Wang, C.; Zhang, R.; Hong, R.; Wang, H.: Attention-enhanced deep reinforcement learning for electric vehicle routing optimization. *IEEE Trans. Transp. Electrific.* pp. 1–1. (2025a)
79. Wang, Y.; Jia, Y.-H.; Chen, W.-N.; Mei, Y.: Soft parameter sharing model for cross-problem generalization in vehicle routing problems. *IEEE Trans. Artific. Intell.*, pp. 1–15. (2025b)
80. Sun, W.; Pei, Z.: Deep reinforcement learning for solving two-echelon capacity vehicle routing problem: an end-to-end method. IEEE Robot. Autom. Lett. **10**(6), 6432–6439 (2025)
81. Khalil, E.; Dai, H.; Zhang, Y.; Dilkina, B.; Song, L.: Learning combinatorial optimization algorithms over graphs. *Adv. Neural Inf. Process. Syst.* vol. 30. (2017)
82. Joshi, C.K.; Laurent, T.; Bresson, X.: An efficient graph convolutional network technique for the travelling salesman problem. arXiv:1906.01227. (2019)
83. Hu, Y.; Zhang, Z.; Yao, Y.; Huyan, X.; Zhou, X.; Lee, W.S.: A bidirectional graph neural network for traveling salesman problems on arbitrary symmetric graphs. Eng. Appl. Artif. Intell. **97**, 104061 (2021)
84. Yu, J.J.Q.; Yu, W.; Gu, J.: Online vehicle routing with neural combinatorial optimization and deep reinforcement learning. IEEE Trans. Intell. Transp. Syst. **20**(10), 3806–3817 (2019)
85. Hu, Y.; Yao, Y.; Lee, W.S.: A reinforcement learning approach for optimizing multiple traveling salesman problems over graphs. Knowl.-Based Syst. **204**, 10624 (2020)
86. Duan, L.; Zhan, Y.; Hu, H.; Gong, Y.; Wei, J.; Zhang, X.; Xu, Y.: Efficiently solving the practical vehicle routing problem: a novel joint learning approach. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 3054–3063. (2020)
87. Wang, Q.; Hao, Y.; Cao, J.: Learning to traverse over graphs with a Monte Carlo tree search-based self-play framework. Eng. Appl. Artif. Intell. **105**, 104422 (2021)
88. Pan, W.; Liu, S.Q.: Deep reinforcement learning for the dynamic and uncertain vehicle routing problem. Appl. Intell. **53**(1), 405–422 (2022)
89. Zhang, R.; Zhang, C.; Cao, Z.; Song, W.; Tan, P.S.; Zhang, J.; Wen, B.; Dauwels, J.: Learning to solve multiple-TSP with time window and rejections via deep reinforcement learning. IEEE Trans. Intell. Transp. Syst. **24**(1), 1325–1336 (2023)
90. Li, K.; Zhang, T.; Wang, R.: Deep reinforcement learning for multiobjective optimization. IEEE Trans. Cybern. **51**(6), 3103–3114 (2021)
91. Wu, H.; Wang, J.; Zhang, Z.: MODRL/D-AM: multiobjective deep reinforcement learning algorithm using decomposition and attention model for multiobjective optimization. In *Artific. Intell. Algorithm. Appl.* pp. 575–589. (2020)
92. Zhang, Y.; Wang, J.; Zhang, Z.; Zhou, Y.: MODRL/D-EL: multiobjective deep reinforcement learning with evolutionary learning for multiobjective optimization. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8. (2021)
93. Zhang, Z.; Wu, Z.; Zhang, H.; Wang, J.: Meta-learning-based deep reinforcement learning for multiobjective optimization problems. IEEE Trans. Neural Netw. Learn. Syst. **34**(10), 7978–7991 (2023)
94. Liu, W.; Wang, R.; Zhang, T.; Li, K.; Li, W.; Ishibuchi, H.; Liao, X.: Hybridization of evolutionary algorithm and deep reinforcement learning for multiobjective orienteering optimization. IEEE Trans. Evol. Comput. **27**(5), 1260–1274 (2023)
95. Wang, Z.; Yao, S.; Li, G.; Zhang, Q.: Multiobjective combinatorial optimization using a single deep reinforcement learning model. IEEE Trans. Cybern. **54**(3), 1984–1996 (2024)
96. Zhou, Y.; Kong, L.; Wang, H.: A multiobjective edge-based learning algorithm for the vehicle routing problem with time windows. Inf. Sci. **715**, 122223 (2025)
97. Jiang, Y.; Wu, Y.; Cao, Z.; Zhang, J.: Learning to solve routing problems via distributionally robust optimization. Proc. AAAI Confer. Artific. Intell. **36**(9), 9786–9794 (2022)
98. Ling, Z.; Zhang, Y.; Chen, X.: A deep reinforcement learning based real-time solution policy for the traveling salesman problem. IEEE Trans. Intell. Transp. Syst. **24**(6), 5871–5882 (2023)
99. Joe, W.; Lau, H.C.: Deep reinforcement learning approach to solve dynamic vehicle routing problem with stochastic customers. In: *Proceedings of the International Conference on Automated Planning and Scheduling***30**, 394–402 (2020)
100. Shi, J.; Gao, Y.; Wang, W.; Yu, N.; Ioannou, P.A.: Operating electric vehicle fleet for ride-hailing services with reinforcement learning. IEEE Trans. Intell. Transp. Syst. **21**(11), 4822–4834 (2020)
101. Delarue, A.; Anderson, R.; Tjandraatmadja, C.: Reinforcement learning with combinatorial actions: an application to vehicle routing. Adv. Neural. Inf. Process. Syst. **33**, 609–620 (2020)
102. Ma, Y.; Hao, X.; Hao, J.; Lu, J.; Liu, X.; Tong, X.; Yuan, M.; Li, Z.; Tang, J.; Meng, Z.: A hierarchical reinforcement learning based optimization framework for large-scale dynamic pickup and delivery problems. (2021a)
103. Liu, Z.; Li, X.; Khojandi, A.: The flying sidekick traveling salesman problem with stochastic travel time: A reinforcement learning approach. Trans. Res. E Logist. Trans. Rev. **164**, 102816 (2022)
104. Chen, X.; Ulmer, M.W.; Thomas, B.W.: Deep Q-learning for same-day delivery with vehicles and drones. Eur. J. Oper. Res. **298**(3), 939–952 (2022)
105. Basso, R.; Kulcsár, B.; Sanchez-Diaz, I.; Qu, X.: Dynamic stochastic electric vehicle routing with safe reinforcement learning. Trans. Res. E: Logist. Transp. Rev. **157**, 102496 (2022)
106. Alqahtani, M.; Hu, M.: Dynamic energy scheduling and routing of multiple electric vehicles using deep reinforcement learning. Energy **244**, 122626 (2022)
107. Mak, S.; Xu, L.; Pearce, T.; Ostroumov, M.; Brintrup, A.: Fair collaborative vehicle routing: a deep multi-agent reinforcement learning approach. Trans. Res. C Emerg. Technol. **157**, 104376 (2023)
108. Zhou, C.; Ma, J.; Douge, L.; Chew, E.P.; Lee, L.H.: Reinforcement learning-based approach for dynamic vehicle routing problem with stochastic demand. Comput. Ind. Eng. **182**, 109443 (2023)
109. Zhang, Y.; Li, M.; Chen, Y.; Chiang, Y.-Y.; Hua, Y.: A constraint-based routing and charging methodology for battery electric vehicles with deep reinforcement learning. IEEE Trans. Smart Grid **14**(3), 2446–2459 (2023)
110. Akkerman, F.; Mes, M.; van Jaarsveld, W.: A comparison of reinforcement learning policies for dynamic vehicle routing problems with stochastic customer requests. Comput. Ind. Eng. **200**, 110747 (2025)
111. Zhang, Z.; Zhang, Z.; Wang, X.; Zhu, W.: Learning to solve travelling salesman problem with hardness-adaptive curriculum. Proc. AAAI Confer. Artific. Intell. **36**(8), 9136–9144 (2022)

112. Bi, J.; Ma, Y.; Wang, J.; Cao, Z.; Chen, J.; Sun, Y.; Chee, Y.M.: Learning generalizable models for vehicle routing problems via knowledge distillation. Adv. Neural. Inf. Process. Syst. **35**, 31226–31238 (2022)
113. Liu, M.; Yuan, Y.; Xu, A.; Deng, T.; Jian, L.: A learning-based artificial bee colony algorithm for operation optimization in gas pipelines. Inf. Sci. **690**, 121593 (2025)
114. Sobhanan, A.; Park, J.; Park, J.; Kwon, C.: Genetic algorithms with neural cost predictor for solving hierarchical vehicle routing problems. Transp. Sci. **59**(2), 322–339 (2025)
115. Shaik, J.; Karri, S.P.K.: Accelerated primal?dual deep reinforcement learning for efficient energy management of hybrid electric vehicles. Arab. J. Sci. Eng. **50**(8), 5525–5540 (2025)
116. Kim, M.; Park, J.; Park, J.: Learning to CROSS exchange to solve min-max vehicle routing problems. In *International Conference on Learning Representations*. (2023)
117. da Costa, P.; Rhuggenaath, J.; Zhang, Y.; Akcay, A.; Kaymak, U.: Learning 2-Opt heuristics for routing problems via deep reinforcement learning. *SN Comput. Sci.* 2(5). (2021)
118. Sui, J.; Ding, S.; Liu, R.; Xu, L.; Bu, D.: Learning 3-opt heuristics for traveling salesman problem via deep reinforcement learning. In *Proceedings of The 13th Asian Conference on Machine Learning*, pp. 1301–1316. (2021)
119. Wu, Y.; Song, W.; Cao, Z.; Zhang, J.; Lim, A.: Learning improvement heuristics for solving routing problems. IEEE Trans. Neural Netw. Learn. Syst. **33**(9), 5057–5069 (2022)
120. Wang, Q.; Hao, Y.; Zhang, J.: Generative inverse reinforcement learning for learning 2-opt heuristics without extrinsic rewards in routing problems. J. King Saud Univ. Comput. Inf. Sci. **35**(9), 101787 (2023)
121. Zheng, J.; He, K.; Zhou, J.: Combining reinforcement learning with lin-kernighan-helsgaun algorithm for the traveling salesman problem. In *AAAI*. (2021)
122. Xin, L.; Song, W.; Cao, Z.; Zhang, J.: NeuroLKH: combining deep learning model with Lin-Kernighan-Helsgaun heuristic for solving the traveling salesman problem. Adv. Neural. Inf. Process. Syst. **34**, 7472–7483 (2021)
123. Zheng, J.; He, K.; Zhou, J.; Jin, Y.; Li, C.-M.: Reinforced Lin-Kernighan-Helsgaun algorithms for the traveling salesman problems. *Knowl.-Based Syst.* 260(C). (2023a)
124. Zheng, J.; Zhong, J.; Chen, M.; He, K.: A reinforced hybrid genetic algorithm for the traveling salesman problem. Comput. Oper. Res. **157**, 106249 (2023)
125. Gambardella, L.M.; Dorigo, M.: Ant-Q: a reinforcement learning approach to the traveling salesman problem. Mach. Learn. Proc. **1995**, 252–260 (1995)
126. Lee, S.: A cooperation online reinforcement learning approach in ant-Q. In *Neural Inf. Process.* pp 487–494 (2006)
127. R., L.C.; Gonzalez B.; J. J.; Orta, J. F. D.; Arrañaga C.; B.A.; Fraire H., H.J.: A new approach to improve the ant colony system performance: learning levels. In *Hybrid Artific. Intell. Syst.* pp. 670–677. (2009)
128. Kamei, K.; Ishikawa, M.: Adaptive decision making in ant colony system by reinforcement learning. In *Neural Inf. Process. Theory Algorithm.* pp. 609–617. (2010)
129. Dries, E.J.; Peterson, G.L.: Scaling ant colony optimization with hierarchical reinforcement learning partitioning. In *Proceedings of the 10th annual conference on Genetic and evolutionary computation*, pp. 25–32. (2008)
130. Nagata, Y.; Kobayashi, S.: A powerful genetic algorithm using edge assembly crossover for the traveling salesman problem. INFORMS J. Comput. **25**(2), 346–363 (2013)
131. Chaves, A.A.; Vianna, B.L.; da Silva, T.T.; Schenekemberg, C.M.: A parallel branch-and-cut and an adaptive metaheuristic to solve the Family Traveling Salesman Problem. Expert Syst. Appl. **238**, 121735 (2024)
132. de Lima Junior, F.C.; de Melo, J.D.; Neto, A.D.D.: Using Q-learning algorithm for initialization of the GRASP metaheuristic and genetic algorithm. In *2007 International Joint Conference on Neural Networks*, pp. 1243–1248. (2007)
133. Alipour, M.M.; Razavi, S.N.; Feizi Derakhshi, M.R.; Balafar, M.A.: A hybrid algorithm using a genetic algorithm and multiagent reinforcement learning heuristic to solve the traveling salesman problem. Neural Comput. Appl. **30**(9), 2935–2951 (2018)
134. dos Santos, J. P.Q.; de Lima, F.C.; Marrocos Magalhaes, R.; de Melo, J.D.; Neto, A.D.D.: A parallel hybrid implementation using genetic algorithm, GRASP and reinforcement learning. In *2009 International Joint Conference on Neural Networks*, pages 2798–2803. (2009)
135. Liu, F.; Zeng, G.: Study of genetic algorithm with reinforcement learning to solve the TSP. *Expert Syst. Appl.*, 36(3, Part 2):6995–7001. (2009)
136. Schenekemberg, C.M.; Chaves, A.A.; Coelho, L.C.; Guimarães, T.A.; Avelino, G.G.: The dial-a-ride problem with private fleet and common carrier. Comput. Oper. Res. **147**, 105933 (2022)
137. Qi, R.; Li, J.-Q.; Wang, J.; Jin, H.; Han, Y.-Y.: QMOEA: A Q-learning-based multiobjective evolutionary algorithm for solving time-dependent green vehicle routing problems with time windows. Inf. Sci. **608**, 178–201 (2022)
138. Chen, M.; Gao, L.; Chen, Q.; Liu, Z.: Dynamic Partial Removal: A Neural Network Heuristic for Large Neighborhood Search. (2020)
139. Gao, L.; Chen, M.; Chen, Q.; Luo, G.; Zhu, N.; Liu, Z.: Learn to Design the Heuristics for Vehicle Routing Problem. arXiv:2002.08539. (2020)
140. Kong, D.; Ma, Y.; Cao, Z.; Yu, T.; Xiao, J.: Efficient neural collaborative search for pickup and delivery problems. *IEEE Trans. Pattern Anal. Mach. Intell.* pp. 1–15. (2024)
141. Hottung, A.; Tierney, K.: Neural large neighborhood search for routing problems. Artif. Intell. **313**(103786), 4 (2022)
142. Wang, C.; Cao, M.; Jiang, H.; Xiang, X.; Zhang, X.: A deep reinforcement learning-based adaptive large neighborhood search for capacitatedelectric vehicle routing problems. *IEEE Trans. Emerg. Top. Comput. Intell.* pp. 1–14. (2024a)
143. Johnn, S.-N.; Darvariu, V.-A.; Handl, J.; Kalcsics, J.: A graph reinforcement learning framework for neural adaptive large neighbourhood search. Comput. Oper. Res. **172**, 106791 (2024)
144. Wang, Z.; Hu, X.; Sun, H.; Xia, W.: ADNS: An adaptive dynamic neighborhood search method guided by joint learning heuristics and corresponding hyperparameters. Appl. Soft Comput. **180**, 113280 (2025)
145. Queiroz dos Santos, J.P.; de Melo, J.D.; Duarte Neto, A.D.; Aloise, D.: Reactive search strategies using reinforcement learning, local search algorithms and variable neighborhood search. Expert Syst. Appl. **41**(10), 4939–4949 (2014)
146. Lopes Silva, M.A.; de Souza, S.R.; Freitas Souza, M.J.; Bazzan, A.L.C.: A reinforcement learning-based multi-agent framework applied for solving routing and scheduling problems. Expert Syst. Appl. **131**, 148–171 (2019)
147. Kalatzantonakis, P.; Sifaleras, A.; Samaras, N.: A reinforcement learning-variable neighborhood search method for the capacitated vehicle routing problem. Expert Syst. Appl. **213**, 118812 (2023)
148. Guo, T.; Mei, Y.; Zhang, M.; Zhao, H.; Cai, K.; Du, W.: Learning-aided neighborhood search for vehicle routing problems. IEEE Trans. Pattern Anal. Mach. Intell. **47**(7), 5930–5944 (2025)
149. Sabar, N.R.; Ayob, M.; Kendall, G.; Qu, R.: A dynamic multiarmed bandit-gene expression programming hyper-heuristic for combinatorial optimization problems. IEEE Trans. Cybern. **45**(2), 217–228 (2015)
150. Soria-Alcaraz, J.A.; Ochoa, G.; Sotelo-Figeroa, M.A.; Burke, E.K.: A methodology for determining an effective subset of

heuristics in selection hyper-heuristics. Eur. J. Oper. Res. **260**(3), 972–983 (2017)
151. Rodríguez-Esparza, E.; Masegosa, A.D.; Oliva, D.; Onieva, E.: A new hyper-heuristic based on adaptive simulated annealing and reinforcement learning for the capacitated electric vehicle routing problem. Expert Syst. Appl. **252**, 124197 (2024)
152. Yao, Y.; Peng, Z.; Xiao, B.: Parallel hyper-heuristic algorithm for multi-objective route planning in a smart city. IEEE Trans. Veh. Technol. **67**(11), 10307–10318 (2018)
153. Chen, X.; Tian, Y.: Learning to perform local rewriting for combinatorial optimization. In *Adv. Neural Inf. Process. Syst.* Vol. 32. (2019)
154. Lu, H.; Zhang, X.; Yang, S.: A learning-based iterative method for solving vehicle routing problems. In *Eighth International Conference on Learning Representations*. (2020)
155. Kallestad, J.; Hasibi, R.; Hemmati, A.; Sörensen, K.: A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems. *Eur. J. Oper. Res.* (2023)
156. Lagos, F.; Pereira, J.: Multi-armed bandit-based hyper-heuristics for combinatorial optimization problems. Eur. J. Oper. Res. **312**(1), 70–91 (2024)
157. Ye, T.; Zhang, Z.; Zhang, Q.; Chen, J.; Wang, J.: Solving multiobjective combinatorial optimization via learning to improve method. IEEE Trans. Emerg. Top. Comput. Intell. **9**(3), 2122–2136 (2025)
158. Qin, W.; Zhuang, Z.; Huang, Z.; Huang, H.: A novel reinforcement learning-based hyper-heuristic for heterogeneous vehicle routing problem. Comput. Ind. Eng. **156**, 107252 (2021)
159. Choong, S.S.; Wong, L.-P.; Lim, C.P.: Automatic design of hyper-heuristic based on reinforcement learning. Inf. Sci. **436–437**, 89–107 (2018)
160. Shang, C.; Ma, L.; Liu, Y.: Green location routing problem with flexible multi-compartment for source-separated waste: A Q-learning and multi-strategy-based hyper-heuristic algorithm. Eng. Appl. Artif. Intell. **121**, 105954 (2023)
161. Cappart, Q.; Moisan, T.; Rousseau, L.-M.; Prémont-Schwarz, I.; Cire, A.A.: Combining reinforcement learning and constraint programming for combinatorial optimization. Proc. AAAI Confer. Artific. Intell. **35**(5), 3677–3687 (2021)
162. Zou, Y.; Hao, J.-K.; Wu, Q.: A reinforcement learning guided hybrid evolutionary algorithm for the latency location routing problem. Comput. Oper. Res. **170**, 106758 (2024)

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.